DataVis: Rust-Native Server-Side Rasterization

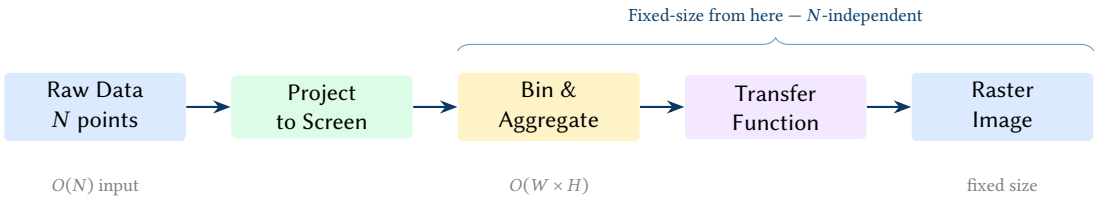
From Datashader's Concepts to a Modern Arrow/Polars Pipeline

Simon Knight, Ph.D.

Adelaide, Australia

March 2026 • Version 1.0.0

Last updated: March 18, 2026



Abstract. Rendering datasets of millions to billions of points demands a fundamentally different approach than conventional vector graphics. Datashader pioneered server-side rasterization in Python, but its reliance on Numba JIT compilation and Dask scheduling limits interactive performance. This report formalises the algorithmic foundations of server-side rasterization and proposes DataVis: a high-performance, Rust-native rendering compiler built on Polars and Apache Arrow. We advance the paradigm beyond simple heatmaps by introducing a Raster-Native Grammar of Graphics, treating the $W \times H$ aggregate as a mathematical tensor. This enables Image-Space Digital Signal Processing (DSP) via FFT band-pass filtering and anisotropic diffusion, as well as multivariate pixel embeddings using UMAP and autoencoder style transfer. Furthermore, we explore advanced scientific computing applications, including Line Integral Convolution (LIC) for vector fields, 3D Spatiotemporal Data Cubes for temporal tracking, and Probabilistic Rendering to natively visualize uncertainty and dataset variance. We demonstrate a streaming pipeline that supports billion-record datasets with sub-second latency while offering unprecedented analytical depth.

Contents

1 Introduction: The Rasterization Imperative **2**

2 Datashader: Lessons and Limitations **2**
2.1 Architecture. 2
2.2 Supported Glyph Types 2
2.3 Limitations Motivating a Rust Successor 2

3 Pixel-Binning Aggregation **3**
3.1 Mathematical Foundation 3
3.2 Monoid Structure and Parallelism 3
3.3 Inner Loop Implementation 3
3.4 Line and Edge Rasterization 4
3.5 Bilinear Aggregation. 4
3.6 Categorical Aggregation 5

4 Transfer Functions and Colormapping **5**
4.1 Linear Normalization 5
4.2 Logarithmic Normalization. 5
4.3 Histogram Equalization 5
4.4 Perceptually Uniform Colormaps 5

5 Edge Bundling Algorithms **6**
5.1 Force-Directed Edge Bundling (FDEB) 6
5.2 Kernel Density Estimation Edge Bundling (KDEEB / Hammer Bundle) 6
5.3 Attribute-Driven Edge Bundling (ADEB) 7

6 Spatial Indexing for Rasterization **7**
6.1 Quadtrees 7
6.2 R-Trees. 8
6.3 Spatial Hashing 8
6.4 Z-order (Morton) Curves 8

7 Tiled Rendering and the Slippy-Map Pyramid **8**
7.1 Tile Coordinates and Web Mercator 8
7.2 Tile Pyramid Structure 9
7.3 On-Demand Tile Rasterization 9

8 Streaming and Out-of-Core Aggregation **10**
8.1 Chunked Streaming 10
8.2 Apache Arrow and Parquet 10
8.3 M4 Time-Series Aggregation 10
8.4 Bilinear Streaming and Chunked Fallbacks 10

9 Anti-Aliasing for Data Visualization **11**
9.1 Bresenham vs. Wu's Algorithm 11
9.2 Bilinear Interpolation for Point Data 11
9.3 Coverage-Weighted Alpha Compositing 11
9.4 Super-Sampling. 11

10 The Rust Ecosystem Landscape **11**
10.1 Data Processing: Polars and Arrow 12
10.2 Geospatial: GeoRust and GeoArrow. 12
10.3 Image Output and Colormapping. 12
10.4 Tile Serving: Martin and Axum 12

11 GPU Acceleration **12**
11.1 wgpu and WGSL Compute Shaders 12
11.2 Alternative: Vulkano and rust-gpu. 13

12 Synthesis: A Rust Implementation Architecture **13**

13 Feasibility Assessment **14**
13.1 Necessity: Why Not Just Use Datashader?. 14
13.2 Ecosystem Readiness. 14
13.3 Risks. 14

13.4 Strategic Fit . . . 14

14 Beyond Raster Tiles: Arrow Flight and Zero-Copy Data Pipelines 15
14.1 Arrow Flight . . . 15
14.2 ADBC (Arrow Database Connectivity) . . . 15
14.3 Flight SQL Tile Server . . . 16
14.4 DuckDB as the Query Engine . . . 16

15 Client-Side Rendering: WebGPU, WASM, and Native UIs 16
15.1 WebGPU in 2026 . . . 16
15.2 GeoArrow + deck.gl Layers . . . 17
15.3 Rust to WASM . . . 17
15.4 Native Desktop: egui + wgpu . . . 17

16 Streaming and Real-Time Visualization 17
16.1 Incremental Aggregate Updates . . . 17

17 Advanced Visualization Techniques 17
17.1 Temporal Animation . . . 17
17.2 Multi-Resolution Level-of-Detail . . . 18
17.3 Contour Extraction . . . 18
17.4 Bivariate Density Maps . . . 18
17.5 Linked Views and Cross-Filtering . . . 18
17.6 Differential Privacy . . . 18
17.7 Comparative Visualization . . . 18
17.8 Feature Readiness Summary . . . 19

18 Visual Intelligence and Spatial Analysis 19
18.1 Hotspot Detection (Getis-Ord Gi*) . . . 19
18.2 Peak Detection and Feature Extraction . . . 19
18.3 GeoJSON Contour Extraction . . . 19

19 Canvas Diagnostics and Automated Tuning 19
19.1 Density and Dynamic Range Analysis . . . 20
19.2 Automated Heuristics for Visual Quality . . . 20

20 Vision: The Full Architecture 20

21 Extended Capabilities and Research Directions 20
21.1 Analytical Overlays . . . 20
21.2 Advanced Rendering Techniques . . . 21
21.3 Interaction and Collaboration . . . 23
21.4 Data and Integration . . . 23
21.5 Performance and Scale . . . 24
21.6 Domain-Specific Extensions . . . 25
21.7 Feature Readiness Summary . . . 26

22 A Raster-Native Grammar of Graphics 27
22.1 Redefining the Grammar for the Pixel Grid . . . 27
22.2 Algebraic Layer Compositing . . . 27
22.3 CLI and Python API as a Rendering Compiler . . . 27

23 Image-Space Signal Processing (DSP) 28
23.1 Frequency Domain Filtering (FFT) . . . 28
23.2 Anisotropic Diffusion and Derivative Fields . . . 28

24 Multivariate Pixel Embeddings and Latent Spaces 29
24.1 Dimensionality Reduction to RGB . . . 29
24.2 Autoencoder Style Transfer and Denoising . . . 29

25 Vector Fields and Continuous Flow Dynamics 30
25.1 Line Integral Convolution (LIC) . . . 30

26 Spatiotemporal Data Cubes ($W \times H \times T$) 30
26.1 Time as a Primary Dimension . . . 30

27 Uncertainty Quantification and Probabilistic Rendering 31
27.1 The Uncertainty Aesthetic . . . 31

28 The Raster-Relational Bridge: The Canvas as a Database **31**
28.1 A Fluent Visual Grammar (Polars-Style API) . . . 31
28.2 The Visual Join: Closing the Analytical Loop . . . 32
28.3 Visual Intelligence and Smart Recommendations . . . 32

29 Governance, Privacy, and Multi-Scale Analysis **32**
29.1 The Privacy Monoid: Differentially Private Binning . . . 32
29.2 Multi-Scale Structural decomposition (Wavelet Analysis). . . . 33
29.3 Visual Assertions: Unit Testing for Data Quality . . . 33
29.4 Hardware-Native Optimization: Apple Silicon and NPU. . . . 33

30 Implementation Results **33**
30.1 Implementation Status . . . 34
30.2 Rendering Pipeline . . . 34
30.3 Performance Optimisations . . . 34
30.4 Example Gallery . . . 35
30.5 Performance Benchmarks . . . 37

31 Complexity Summary **37**

KDE Kernel Density Estimation . . . 13
LUT Look-Up Table . . . 6
CDF Cumulative Distribution Function . . . 5
JIT Just-In-Time . . . 2
DOM Document Object Model . . . 2
DP Differential Privacy . . . 21

1 Introduction: The Rasterization Imperative

Vector graphics are the natural output of most visualization pipelines. An edge list becomes an SVG `<path>`; a scatter plot becomes thousands of SVG `<circle>` elements. This model degrades badly at scale. The browser Document Object Model (DOM) cannot efficiently render more than a few thousand overlapping elements; alpha compositing in SVG is per-element (painter's model) rather than additive, which prevents density estimation; and no SVG-level operation corresponds to "show me how many points land in each pixel".

Insight: *A 4 K raster is 32 MB regardless of dataset size. An SVG with 20 K paths exceeds 50 MB and locks the browser DOM.*

The solution, formalized by Cottam et al. as "Abstract Rendering" [1] and popularized by Datashader [2], is to move rendering to the server and return a raster image. The pipeline has five stages (fig. 1):

1. **Projection** — map data coordinates to screen coordinates.
2. **Aggregation** — accumulate statistics into a $W \times H$ grid.
3. **Transformation** — apply a transfer function (log, equalize, …).
4. **Colormapping** — convert scalar grid to RGBA pixels.
5. **Embedding** — deliver the image to the client.

Only stage 2 touches all N input records. Every subsequent stage operates on the fixed-size $W \times H$ aggregate — making the pipeline $O(N)$ in data size and $O(WH)$ in memory, independent of the relationship between N and WH.

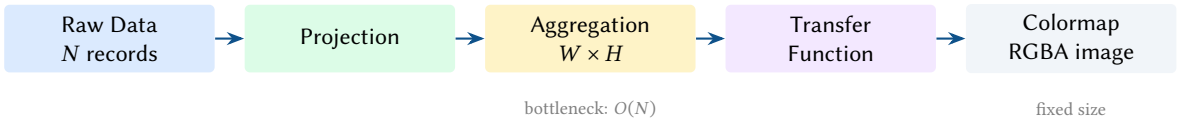


Figure 1. The server-side rasterization pipeline. Only the Aggregation stage reads all N input records. All downstream stages are $O(WH)$.

2 Datashader: Lessons and Limitations

Datashader [2] is the canonical Python implementation of server-side rasterization. Understanding its architecture and failure modes is essential context for a successor system.

2.1 Architecture

Datashader's five-stage pipeline (Projection → Aggregation → Transformation → Colormapping → Embedding) is the reference model for this report. The aggregation kernels are written in Python but compiled via Numba's Just-In-Time (JIT) with `nopython=True` and `nogil=True`, achieving near-C performance on hot loops. Dask [3] provides distributed DataFrames for out-of-core processing. The output aggregate is an xarray `DataArray`; transfer functions (`shade()`, `spread()`, `dynspread()`) produce PIL images.

2.2 Supported Glyph Types

Datashader can rasterize points, lines (with Xiaolin Wu anti-aliasing since PR #916), filled areas, triangle meshes (trimesh), curvilinear grids (quadmesh), raster regridding, and polygons (via SpatialPandas ragged arrays). Categorical reductions (`count_cat`) produce 3D aggregate arrays with per-category colour mixing.

2.3 Limitations Motivating a Rust Successor

- **JIT warmup latency.** Numba's first compilation of any new data type, column type, or reduction combination takes 1–5 s. When using the Dask distributed scheduler (not threaded), compiled functions are *not cached across workers*, adding 0.5–1 s per render (GitHub issue #684).
- **Numba version sensitivity.** Performance regressions of up to 3× have been observed across Numba versions (e.g., 0.48 → 0.49).
- **GPU coverage gaps.** The cuDF GPU path supports only points, lines, and areas. Trimesh, quadmesh, polygon, and bundling are CPU-only. These limitations are poorly documented and cause user confusion.
- **Polygon rendering is slow.** SpatialPandas ragged-array polygon rasterization is substantially slower than point/line rendering, even with Numba.
- **Geo reprojection overhead.** Datashader is projection-agnostic; geographic reprojection must be reapplied on every zoom/pan update via the GeoViews layer, creating significant interactive latency.
- **No per-record interactivity.** Once data is aggregated, individual record identity is lost. This is inherent to the approach (not a bug), but the lack of a standard "drill-down" mechanism is a common user complaint.
- **Dependency rot.** DataShape was abandoned upstream and had to be vendored in Datashader 0.16. Dask and Pillow were only made optional in 0.17. The ecosystem coupling to HoloViews/Bokeh/Panel creates a steep learning curve.

Insight: *Numba JIT warmup on every distributed worker makes sub-second interactive tiles unachievable in Datashader. A compiled language eliminates this entirely.*

Design Decision 1: Design Opportunity

A Rust implementation eliminates JIT warmup, removes the Numba/Dask/xarray dependency chain, and delivers predictable latency. Arrow provides the same zero-copy semantics Datashader gets from Numba memory views, but across language boundaries through the Arrow C Data Interface.

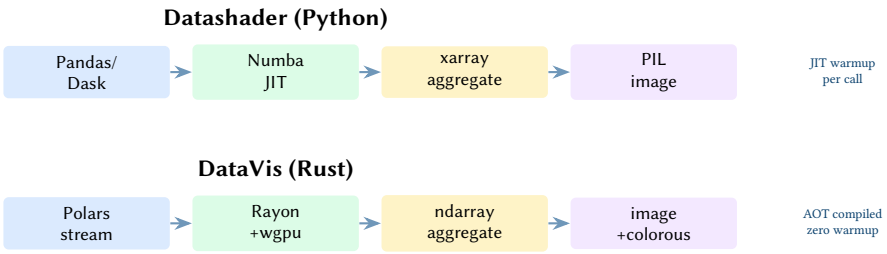


Figure 2. Datashader vs. DataVis: corresponding pipeline stages. The Rust stack replaces JIT compilation with ahead-of-time compilation, Dask with Polars streaming, and the PIL/xarray output with the `image`/`colorous` crates.

3 Pixel-Binning Aggregation

3.1 Mathematical Foundation

Let the canvas be a grid of $W \times H$ bins. Each bin b_{xy} covers a rectangular region of data space $[x_{\min}, x_{\max}) \times [y_{\min}, y_{\max})$. Given a data point (d_x, d_y) with value v, the bin indices are:

$$i = \left\lfloor \frac{d_x - x_{\min}}{x_{\max} - x_{\min}} \cdot W \right\rfloor, \qquad j = \left\lfloor \frac{d_y - y_{\min}}{y_{\max} - y_{\min}} \cdot H \right\rfloor$$

The aggregate array $A[i, j]$ accumulates a *reduction* over all points mapping to bin (i, j). Common reductions are:

Reduction	Identity	Combine	Use case
`count`	0	$a + b$	density heatmaps
`sum`	0	$a + b$	cumulative values
`mean`	$(0, 0)$	$(s_a + s_b,\ n_a + n_b)$; final: s/n	average per pixel
`min`	$+\infty$	$\min(a, b)$	minimum value
`max`	$-\infty$	$\max(a, b)$	maximum value
`any`	`false`	$a \vee b$	boolean presence

3.2 Monoid Structure and Parallelism

The critical algebraic property exploited by Datashader [2] and similar systems is that all efficient reductions form a *monoid* $(S, \oplus, \mathbf{0})$. In plain terms, a monoid is just a merge rule with two guarantees: you can combine any two partial results of the same kind, and there is a neutral "empty" value that changes nothing when merged. Formally, $\oplus$ must be associative, so $(a \oplus b) \oplus c = a \oplus (b \oplus c)$, and $\mathbf{0}$ must be an identity, so $a \oplus \mathbf{0} = a$.

Insight: *Monoid reductions decompose trivially over data partitions: compute partial aggregates in parallel, then merge. This is the theoretical basis for distributed and out-of-core rasterization.*

For rasterization, the practical meaning is simple: each worker can build its own partial canvas, and those canvases can later be merged in any order without changing the answer. For `count` and `sum`, the monoid combine is simply +, so any partition of the N input records produces partial aggregate arrays that can be element-wise summed to recover the global result. For `mean`, one must carry the partial $(sum, count)$ pair; for `min`/`max`, the element-wise min/max. This makes these reductions composable without revisiting raw data.

3.3 Inner Loop Implementation

The aggregation inner loop is the performance bottleneck. In Datashader the loop is written in Python but compiled to native code with Numba's JIT compiler [2]:

Listing 1. Conceptual inner loop for point aggregation (count reduction).

```
@ngjit  # Numba JIT with no-GIL, no-Python-overhead
def _accumulate_count(xs, ys, out, x_range, y_range, W, H):
    x_min, x_max = x_range
    y_min, y_max = y_range
    for k in range(len(xs)):
        x, y = xs[k], ys[k]
```

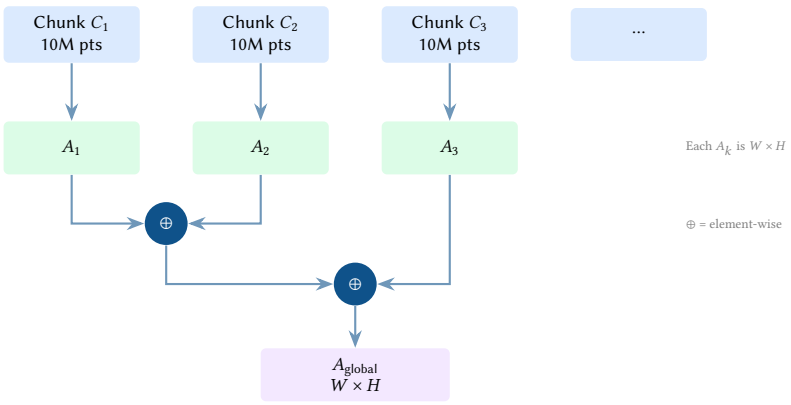


Figure 3. Monoid merge for parallel/streaming aggregation. Partial aggregates from independent chunks are element-wise combined. Associativity ensures correctness regardless of partition order or granularity.

```
        if x_min <= x < x_max and y_min <= y < y_max:
            i = int((x - x_min) / (x_max - x_min) * W)
            j = int((y - y_min) / (y_max - y_min) * H)
            out[j, i] += 1
```

In a Rust implementation this loop becomes a simple nested iterator over columnar Arrow arrays, dispatched with Rayon for data-parallel execution:

Listing 2. Rust pixel-binning kernel using Rayon parallel iterator.

```
pub fn bin_points(xs: &[f64], ys: &[f64],
                  canvas: &mut Array2<u32>,
                  x_range: (f64, f64), y_range: (f64, f64)) {
    let (w, h) = (canvas.ncols(), canvas.nrows());
    let (x_min, x_max) = x_range;
    let (y_min, y_max) = y_range;

    // Parallel fold into per-thread partial canvases, then merge
    let result = xs.par_iter().zip(ys.par_iter())
        .fold(|| Array2::<u32>::zeros((h, w)), |mut acc, (&x, &y)| {
            if x >= x_min && x < x_max && y >= y_min && y < y_max {
                let i = ((x - x_min) / (x_max - x_min) * w as f64) as usize;
                let j = ((y - y_min) / (y_max - y_min) * h as f64) as usize;
                acc[[j.min(h-1), i.min(w-1)]] += 1;
            }
            acc
        })
        .reduce(|| Array2::zeros((h, w)), |a, b| a + b);
    *canvas = result;
}
```

3.4 Line and Edge Rasterization

For edge data, each edge (p_1, p_2) must be rasterized onto the bin grid. The canonical algorithm is Bresenham's line algorithm [4], which iterates over all pixels on a line segment using only integer arithmetic. For each traversed bin (i, j), the reduction is applied. Alpha-composited rendering adds a fractional coverage weight (Wu's algorithm, section 9).

3.5 Bilinear Aggregation

Standard point aggregation (snapping to the nearest pixel) is equivalent to nearest-neighbor interpolation. For sparse datasets or high zoom levels, this produces visible "stair-stepping" and blocky artifacts.

Bilinear aggregation distributes each point's contribution to the four surrounding pixel centers $(i, j), (i+1, j), (i, j+1), (i+1, j+1)$ based on fractional weights:

$$w_{00} = (1 - \Delta x)(1 - \Delta y), \quad w_{10} = \Delta x(1 - \Delta y), \quad w_{01} = (1 - \Delta x)\Delta y, \quad w_{11} = \Delta x \Delta y$$

where $\Delta x, \Delta y \in [0, 1)$ are the fractional pixel offsets. This ensures that the total weight $\sum w_{ij} = 1.0$ is preserved (mass conservation) while producing a smooth density field.

Insight:
Bilinear aggregation is the point-data equivalent of Wu's line algorithm: it is a low-pass filter applied at the moment of rasterization, preventing aliasing at the source.

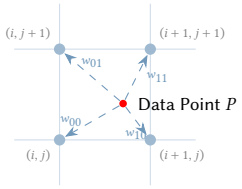


Figure 4. Bilinear weight distribution. A single data point P distributes its unit contribution across four adjacent pixel centers. This prevents aliasing artifacts as points move smoothly across the grid.

3.6 Categorical Aggregation

In addition to scalar reductions, DataVis supports *categorical aggregation*. Given a dataset with a discrete category column (e.g., vehicle type, city, or status code), the aggregator produces a separate $W \times H$ canvas for each unique category found in the viewport.

$$A_{\text{cat}}[i, j] = \text{reduce}(\{d \in \text{records} : \text{bin}(d) = (i, j) \text{ and } \text{category}(d) = \text{cat}\})$$

This multi-layer aggregate is typically rendered as a color-coded heatmap, where each pixel's color is a weighted blend of the category colormaps based on their relative density.

: Aggregation Before Rendering

Aggregate raw records into a fixed $W \times H$ array before applying any visual transformation. This decouples dataset scale from image scale and ensures all downstream operations are $O(WH)$.

4 Transfer Functions and Colormapping

After aggregation, $A[i, j]$ contains a scalar (e.g., a count) that may span many orders of magnitude. A *transfer function* $T : \mathbb{R} \to [0, 1]$ maps this scalar to a normalized value suitable for colormapping.

4.1 Linear Normalization

The simplest transfer function is linear:

$$T_{\text{lin}}(v) = \frac{v - A_{\min}}{A_{\max} - A_{\min}}$$

Linear normalization is appropriate when the data has bounded, roughly uniform dynamic range. It is poorly suited to density maps where a few high-density pixels dominate: the majority of bins map to near-zero and appear uniform.

4.2 Logarithmic Normalization

Logarithmic normalization compresses high dynamic range:

$$T_{\log}(v) = \frac{\log(1 + v) - \log(1 + A_{\min})}{\log(1 + A_{\max}) - \log(1 + A_{\min})}$$

The $+1$ offset ensures $T_{\log}(0) = 0$. Log normalization is the default for density heatmaps: a bin with 10 points and one with 10 000 points map to a ratio of $\log(10)/\log(10^4) = 0.25$ rather than 0.001.

4.3 Histogram Equalization

Insight: *Logarithmic normalization is almost always the right choice for count-based density maps. Linear normalization makes the common case (sparse bins) invisible.*

Histogram equalization redistributes values so each output level is occupied by an equal fraction of bins. Let $F(v)$ be the empirical Cumulative Distribution Function (CDF) of non-zero values in A. Then:

$$T_{\text{eq}}(v) = F(v) = \frac{|\{(i, j) : A[i, j] \leq v\}|}{|\{(i, j) : A[i, j] > 0\}|}$$

This is implemented by sorting the unique values, computing their rank, and interpolating. Equalization maximizes perceptual contrast but can over-enhance noise and obscure monotonic relationships. It is most useful when the data distribution is highly non-uniform and the exact magnitude is less important than relative structure.

4.4 Perceptually Uniform Colormaps

Once $T(v) \in [0, 1]$, it is mapped to an RGBA color via a colormap $C : [0, 1] \to \text{RGBA}$. Perceptual uniformity requires that equal increments in T produce equal increments in perceived lightness. This is formalized using

the CIECAM02 color appearance model; the viridis colormap [5] (designed by van der Walt and Smith, 2015) achieves near-perfect uniformity in the CAM02-UCS perceptual space.

> **Design Decision 2: Colormap Selection**
>
> Default to **viridis** or **inferno** (perceptually uniform, colorblind-safe). Use **diverging** colormaps (e.g. coolwarm) only when the data has a meaningful centre. Never use jet/rainbow: it introduces false features due to non-monotonic luminance [6].

A colormap is most efficiently stored as a Look-Up Table (LUT): a pre-sampled array of 256 RGBA values. The normalized scalar is multiplied by 255 and looked up:

$$\text{RGBA}(v) = \text{LUT}\big[\lfloor T(v) \cdot 255 \rfloor\big]$$

For $W \times H$ pixel grids this lookup costs $O(WH)$ with memory access pattern that is cache-friendly when the LUT fits in L1 cache.

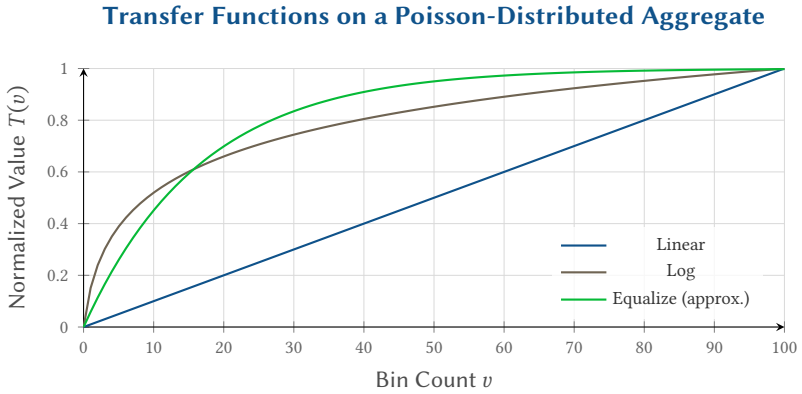


Figure 5. Comparison of transfer functions. Log normalization compresses extreme values and expands the sparse low-count region, which dominates most density maps.

5 Edge Bundling Algorithms

Graph edge bundling reduces visual clutter in dense node-link diagrams by routing nearby edges along shared paths, revealing high-level flow structure. Three algorithms are in common use.

5.1 Force-Directed Edge Bundling (FDEB)

Holten and van Wijk [7] model each edge as a flexible spring subdivided into n control points. Pairs of edges attract each other with a force proportional to their *compatibility* $c(e_1, e_2) \in [0, 1]$, which is the product of angle, scale, position, and visibility scores.

Complexity. Compatibility must be computed for every pair, giving $O(E^2)$ pre-processing where E is the number of edges. Each iteration then applies $O(E \cdot n)$ spring forces. With typical $n = 16$–64 control points and 50–200 iterations, the total cost is $O(E^2 + E \cdot n \cdot I)$. This limits FDEB to at most a few thousand edges in practice.

5.2 Kernel Density Estimation Edge Bundling (KDEEB / Hammer Bundle)

Insight:
FDEB is elegant and produces smooth bundles, but its $O(E^2)$ compatibility matrix makes it impractical beyond ~5 000 edges. Use KDEEB for large graphs.

Hurter, Ersoy, and Telea [8] reframe bundling as an image-space operation, avoiding pairwise compatibility entirely. The algorithm is:

1. **Sample** each edge at m uniform control points to produce a point cloud.
2. **Splat** each point onto a density grid D using a Gaussian kernel K_σ of bandwidth σ:

$$D[i, j] = \sum_{\text{pts}} K_\sigma(x_i - p_x,\ y_j - p_y), \quad K_\sigma(\Delta) = \exp\left(-\frac{|\Delta|^2}{2\sigma^2}\right)$$

3. **Advect** each control point toward the density gradient ∇D, pulling edges toward high-density regions:

$$p_k \leftarrow p_k + \alpha \cdot \nabla D(p_k)$$

4. **Smooth** control points with a Laplacian pass to eliminate artifacts:

$$p_k \leftarrow \lambda p_{k-1} + (1 - 2\lambda) p_k + \lambda p_{k+1}$$

5. **Decrease** bandwidth: $\sigma \leftarrow \sigma \cdot \delta$ (typically $\delta = 0.7$).
6. Repeat steps 2–5 for $I = 8$–10 iterations.

Complexity. Splatting $m \cdot E$ points onto a $W \times H$ grid costs $O(mE)$ per iteration (kernel support is bounded). Advection is $O(mE)$. Total cost is $O(I \cdot m \cdot E)$ — linear in E, making it tractable for millions of edges. Compared to FDEB, KDEEB is typically 16× faster and 6× faster than geometry-based GBEB [8].

The Datashader library provides this algorithm as `hammer_bundle` (attributed to Ian Calvert [9]), parameterised by `initial_bandwidth` (default 0.05) and `decay` (default 0.7). The output is a DataFrame of control-point coordinates with NaN separators between edges, directly suitable for rasterization.

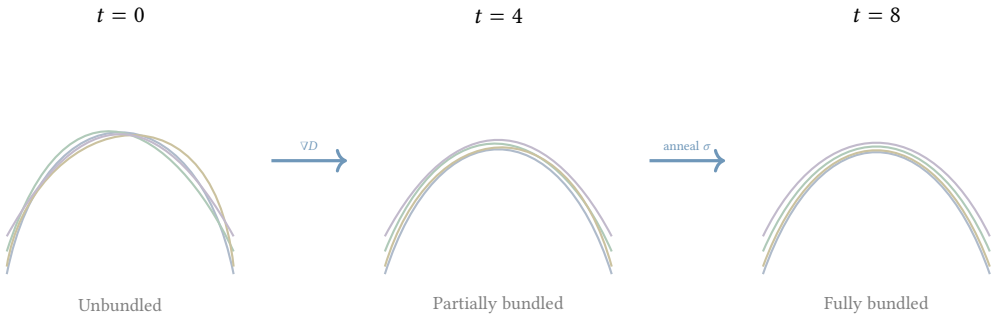


Figure 6. KDEEB edge bundling progression. Control points are advected toward density maxima (∇D), converging into tight bundles. Bandwidth annealing ($\sigma \cdot \delta$) refines the bundle structure across iterations.

Algorithm 1 Hammer Bundle (KDEEB) — core iteration

Input: edges $\mathcal{E}$, initial bandwidth σ_0, decay δ, iterations I, step size α
Sample each edge $e \in \mathcal{E}$ into m control points $\{p_k^e\}$
for $t = 1$ to I **do**
    $D \leftarrow \mathbf{0}_{W \times H}$
    for all control points p **do**
        Splat Gaussian K_{σ_t} centred at p onto D
    end for
    for all control points p **do**
        $p \leftarrow p + \alpha \cdot \nabla D(p)$ ▷ density gradient advection
        Apply Laplacian smoothing to p's neighbours
    end for
    $\sigma_{t+1} \leftarrow \sigma_t \cdot \delta$ ▷ bandwidth annealing
end for
return sampled edge paths $\{p_k^e\}$

5.3 Attribute-Driven Edge Bundling (ADEB)

Peysakhovich, Hurter, and Telea [10] extend KDEEB by weighting the density map with edge attributes (e.g., traffic volume, time of day). Instead of a uniform Gaussian splat, each control point splatted with weight w_e produces a *weighted* density field. Advection then pulls edges toward attribute-coherent clusters. Complexity remains $O(I \cdot m \cdot E)$.

: Edge Bundling at Scale

For graphs with more than ~5 000 edges, use KDEEB (`hammer_bundle`) rather than FDEB. The $O(IE)$ scaling makes it practical for millions of connections, and the image-space implementation maps directly onto GPU compute shaders or SIMD-vectorized CPU code.

6 Spatial Indexing for Rasterization

Pixel-binning does not require a spatial index — the bin computation is $O(1)$ per point via the affine mapping in section 3. Spatial indices become relevant for two related problems: (a) *range queries* during interactive pan/zoom (which points fall in the current viewport?), and (b) *nearest-neighbour lookup* during edge bundling splatting.

6.1 Quadtrees

A quadtree [11] partitions 2D space by recursively subdividing each node into four equal quadrants. Insertion is $O(\log_4 N)$ amortised. Range queries cost $O(k + \log N)$ where k is the number of returned points. **Limitation**: trees become unbalanced for clustered data, degrading worst-case insertion to $O(N)$ for points on a line.

6.2 R-Trees

The R-tree [12] stores axis-aligned minimum bounding rectangles at each node, packing points into leaves. It supports range queries in $O(\sqrt{N})$ expected time and is preferred for geospatial queries involving complex extents. R-trees outperform quadtrees for nearest-neighbour search [12].

6.3 Spatial Hashing

Spatial hashing (grid hashing) assigns each point to a fixed-size cell via $(i, j) = (\lfloor x/c \rfloor, \lfloor y/c \rfloor)$ for cell size c, then uses a hash map. Lookup is $O(1)$ expected.

6.4 Z-order (Morton) Curves

Morton codes [13] interleave the binary representations of x and y coordinates to produce a 1D key that preserves 2D spatial locality. Sorting points by Morton code and processing them in order improves cache utilization dramatically for large point clouds. This technique is used in columnar formats like GeoParquet and in spatial databases.

Insight:
For pixel-binning where the cell size equals the bin size, spatial hashing degenerates to the direct affine mapping of section 3 — no index needed at all.

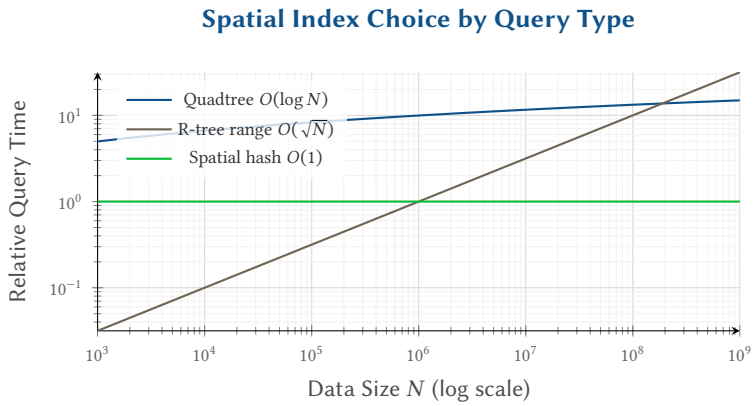


Figure 7. Indicative scaling of spatial index query time. For pixel-binning (single-cell lookup), spatial hashing dominates. For range queries, quadtrees and R-trees are preferable.

Design Decision 3: Spatial Index Selection

- **Pixel-binning**: direct affine mapping — no index.
- **Viewport range queries**: quadtree (simple, balanced data) or R-tree (clustered/geospatial data).
- **KDE splatting (edge bundling)**: fixed grid with Morton-ordered traversal for cache efficiency.
- **Out-of-core streaming**: pre-sort by Morton code, process spatially coherent chunks.

7 Tiled Rendering and the Slippy-Map Pyramid

Choosing the optimal output resolution for a dataset involves a trade-off between visual detail and memory usage. DataVis implements a resolution recommendation engine based on *points-per-pixel (PPP)* targets:

- **Screen Display**: Targets 3 points/pixel for a high-density, continuous look on interactive displays.
- **Print/Export**: Targets 10 points/pixel for a finer, more granular representation suitable for high-DPI output.
- **Tile Serving**: Always uses a fixed 512×512 resolution to match standard slippy-map protocols.

To prevent out-of-memory errors on large canvases, the system enforces a strict **512 MB memory budget** for the RGBA pixel buffer (approx. 64 million pixels). If the ideal resolution based on the target PPP would exceed this budget, the width and height are automatically scaled down while preserving the data aspect ratio.

7.1 Tile Coordinates and Web Mercator

The Web Mercator projection (EPSG:3857) treats the Earth as a sphere and applies the Mercator transformation to latitude ϕ:

$$y_{\text{merc}} = \ln\left(\tan\left(\frac{\pi}{4} + \frac{\phi}{2}\right)\right)$$

The world is clipped to $\phi \in [-85.051^\circ, 85.051^\circ]$ (the latitude at which y_{merc} equals π), producing a square coordinate space.

At zoom level z, the world is divided into $2^z \times 2^z$ tiles. The tile indices for longitude λ and latitude ϕ are:

Insight:
By dynamically adjusting resolution based on the dataset size and the intended use case, DataVis ensures that heatmaps are neither oversaturated (too few pixels) nor sparse (too many pixels), while maintaining a predictable memory footprint.

$$x_{\text{tile}} = \left\lfloor \frac{\lambda + 180}{360} \cdot 2^z \right\rfloor \tag{1}$$

$$y_{\text{tile}} = \left\lfloor \frac{1 - \frac{\ln(\tan\phi + \sec\phi)}{\pi}}{2} \cdot 2^z \right\rfloor \tag{2}$$

Tiles are addressed by the URL scheme `/{z}/{x}/{y}.png`. Each zoom level doubles resolution: zoom 0 is the whole world in one tile; zoom 20 gives sub-metre resolution with $2^{40} \approx 10^{12}$ tiles.

7.2 Tile Pyramid Structure

The tile pyramid (fig. 8) is a quad-tree over tiles. A tile server receives requests `(z, x, y)` and returns a pre-rendered or on-demand computed PNG.

Insight: *Only the tiles visible in the current viewport need to be rendered — at zoom z, this is at most a few tens of tiles regardless of total pyramid size.*

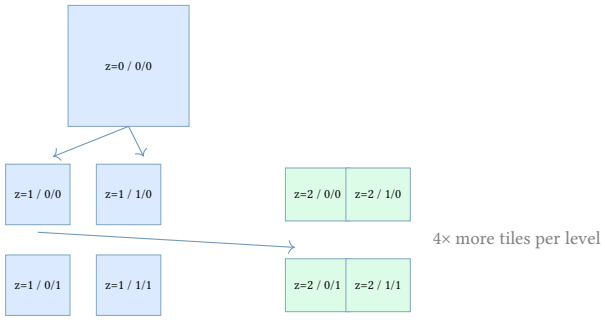


Figure 8. Tile pyramid: each zoom level quadruples the tile count. At zoom z, the world contains 4^z tiles of 256×256 pixels.

7.3 On-Demand Tile Rasterization

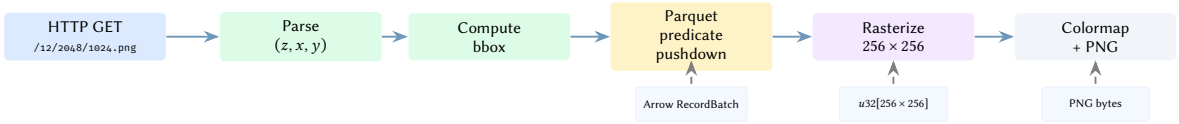


Figure 9. On-demand tile rasterization request flow. The Parquet predicate pushdown skips row groups outside the tile bounding box, minimising I/O.

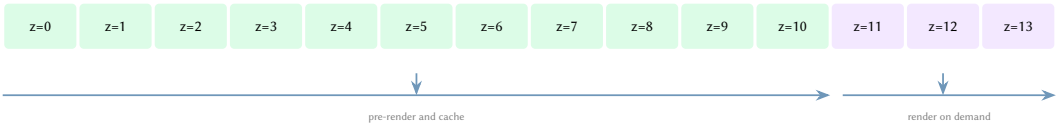


Figure 10. Suggested tile-cache policy by zoom level. Low zoom levels have small, reusable tile sets and benefit from pre-rendering; higher zoom levels are too numerous to cache exhaustively and should be rendered on demand.

A tile server integrates the binning pipeline with the tile coordinate system:

1. Receive request (z, x, y).
2. Compute the geographic bounding box $(\lambda_{\min}, \phi_{\min}, \lambda_{\max}, \phi_{\max})$ of tile (x, y) at zoom z.
3. Query the spatial index (or scan a Parquet partition) for all records within the bounding box.
4. Run the rasterization pipeline with $W = H = 256$ (or 512).
5. Return the PNG.

Pre-computing (caching) tiles at fixed zoom levels is called a *tile cache*. At the lowest zoom levels (0–5) this is essential; at high zoom levels (15+) on-demand rendering is preferred because the tile count is astronomically large.

: Tile Cache Strategy

Pre-render and cache zoom levels 0–10. Render on demand for zoom 11+. Use Parquet files spatially partitioned by GeoHash or Hilbert curve to ensure that tile bounding-box queries touch the minimum number of file partitions.

8 Streaming and Out-of-Core Aggregation

When the dataset exceeds available RAM, the aggregation stage must process data in chunks without materialising the full dataset.

8.1 Chunked Streaming

Given the monoid structure of aggregation reductions (section 3), a chunk-wise streaming protocol is straightforward:

1. Initialise global aggregate $A_{\text{global}} \leftarrow \mathbf{0}_{W \times H}$.
2. For each chunk C_k of B records read from disk:
   (a) Compute partial aggregate $A_k = \text{reduce}(C_k)$.
   (b) Merge: $A_{\text{global}} \leftarrow A_{\text{global}} \oplus A_k$.
3. Apply transfer function and colormap to A_{global}.

Peak memory is $O(WH + B)$ — the aggregate array plus one chunk. For a 4096×4096 `f32` aggregate, this is 64 MB; a chunk of 10 million (x, y) pairs in `f64` is 160 MB. Total peak memory is thus well under 300 MB for any dataset size.

8.2 Apache Arrow and Parquet

Insight: *The monoid decomposition makes streaming aggregation trivially correct. Peak memory is determined by the canvas size and chunk size — not by N.*

Apache Arrow [14] defines a columnar in-memory format with zero-copy IPC semantics. Parquet provides a compressed columnar storage format with predicate pushdown: when reading a Parquet file for a tile bounding box query, the row group statistics (min/max per column) allow entire row groups to be skipped if they cannot intersect the viewport.

Polars [15] implements a streaming query engine over Arrow/Parquet that applies this optimization automatically via its lazy evaluation model:

Listing 3. Polars streaming aggregation over a large Parquet dataset.

```
import polars as pl

result = (
    pl.scan_parquet("data/*.parquet")          # lazy: nothing read yet
      .filter((pl.col("x").is_between(x_min, x_max)) &
              (pl.col("y").is_between(y_min, y_max)))
      .with_columns([                          # bin assignment
          ((pl.col("x") - x_min) / (x_max - x_min) * W)
              .cast(pl.Int32).clip(0, W-1).alias("ix"),
          ((pl.col("y") - y_min) / (y_max - y_min) * H)
              .cast(pl.Int32).clip(0, H-1).alias("iy"),
      ])
      .group_by(["ix", "iy"])
      .agg(pl.len().alias("count"))
      .collect(streaming=True)               # streaming=True: O(WH) RAM
)
```

Polars's streaming engine chunks the Parquet read, applies the filter and bin computation per chunk, and merges partial group-by aggregates. The final `collect` returns only the WH unique bins.

8.3 M4 Time-Series Aggregation

For time-series visualizations (line charts), Jugel et al. [16] propose the M4 aggregation, which retains for each pixel column (time bin) the four values: first, last, min, max. This is the minimal information needed to reconstruct a visually lossless line chart. Formally:

$$M4(C_k) = \{(\text{first}, \text{last}, \text{min}, \text{max}) \ : \ \text{for each pixel column } k\}$$

M4 is a monoid: $M4(A) \oplus M4(B)$ simply takes the appropriate extremes across the two partial results. This makes it suitable for streaming aggregation.

8.4 Bilinear Streaming and Chunked Fallbacks

While standard binning is naturally streaming-friendly, bilinear aggregation (section 3) requires an `explode` operation to distribute each point to its four neighbors. In the Polars query engine, this transformation can cause the streaming executor to fall back to in-memory execution if the intermediate "exploded" dataset (now $4N$ records) exceeds available memory.

To maintain a constant memory profile for bilinear aggregation, DataVis implements a manual chunked streaming strategy:

1. Split the source `LazyFrame` into batches of B records (e.g., $B = 500,000$).
2. For each batch, execute the bilinear aggregation pipeline (`explode`, `group_by`, `collect`).
3. Merging partial aggregate arrays: A_{global} += A_{batch}.

Insight: *By manually batching the source before the explode operation, we guarantee that the peak intermediate memory is bounded by $O(4B + WH)$, preventing OOM errors on billion-record datasets even when the underlying query engine's streaming mode is unavailable for complex plans.*

: Out-of-Core Rasterization

Use chunked streaming over Parquet with predicate pushdown for spatial filtering. Aggregate reductions are monoids; no global sort or shuffle is required. For tile servers, spatially partition Parquet files by GeoHash to co-locate tile-coherent data, minimising I/O per tile request.

9 Anti-Aliasing for Data Visualization

Aliasing arises when the rasterized representation of a sub-pixel feature (a thin line, a small point) assigns full coverage to a pixel or none. In data visualization, anti-aliasing has a dual purpose: visual quality *and* accurate density representation.

9.1 Bresenham vs. Wu's Algorithm

Bresenham's algorithm [4] is exact but aliased: every traversed pixel receives a count increment of 1, regardless of the line's distance from the pixel centre. Wu's algorithm [17] assigns fractional coverage proportional to closeness:

Given a line from (x_0, y_0) to (x_1, y_1) and the current pixel (i, j), let d be the perpendicular distance from the pixel centre to the ideal line. Wu's algorithm writes:

$$A[j_{\text{floor}}, i] \mathrel{+}= (1 - \{y\}), \qquad A[j_{\text{floor}} + 1, i] \mathrel{+}= \{y\}$$

where $\{y\} = y - \lfloor y \rfloor$ is the fractional part of the exact intersection. This distributes the unit contribution between two adjacent pixels, antialiasing at the cost of one additional write per step.

Insight: *Wu's algorithm is the natural anti-aliasing choice for density aggregation: the fractional weights are additive contributions to the reduction monoid, preserving total mass.*

9.2 Bilinear Interpolation for Point Data

For point datasets, anti-aliasing is achieved via bilinear aggregation (section 3). By distributing each point's unit weight to the four surrounding pixels, we eliminate the "popping" effect seen in nearest-neighbor binning as points cross pixel boundaries. This produces a C^0 continuous density surface, which is essential for downstream analysis such as contour extraction or peak detection (section 18).

9.3 Coverage-Weighted Alpha Compositing

For additive blending (the standard for density heatmaps), each point or edge contributes a coverage $\gamma \in [0, 1]$ to the aggregate:

$$A[i, j] \mathrel{+}= \gamma \cdot w$$

where w is an optional scalar weight. This extends seamlessly to Gaussian splatting (coverage = $K_\sigma(d)$), producing soft point glyphs at sub-pixel scales. At low zoom levels many points overlap; their individual Gaussian profiles sum to a smooth density field.

9.4 Super-Sampling

An alternative is to rasterize at $k\times$ resolution and then downsample by averaging. This is $O(k^2)$ more expensive but requires no algorithmic change. For $k = 2$ (4× super-sampling) it is equivalent to MSAA-4x in graphics. In data visualization this is rarely the right trade-off: the direct sub-pixel weighting of Wu or Gaussian splatting achieves equivalent quality at $O(1)$ overhead.

Design Decision 4: Anti-Aliasing Strategy

For line rasterization: use Wu's algorithm rather than Bresenham; the fractional coverage preserves aggregate mass. For point/glyph rendering: use Gaussian splats with bandwidth matched to screen pixel size. Avoid super-sampling for server-side pipelines — the per-point weighting approach has zero memory overhead.

10 The Rust Ecosystem Landscape

No production Rust library replicates Datashader's full pipeline at comparable maturity. However, all necessary building blocks exist as mature, actively maintained crates. This section surveys them.

10.1 Data Processing: Polars and Arrow

Polars [15] is a Rust-native DataFrame library using Apache Arrow as its in-memory format. Key features for rasterization:

- **Streaming engine** (production-ready since v1.31): processes data in fixed-size batches (~100 MB), enabling true out-of-core computation on a single node. Reportedly 3–7× faster than the in-memory engine for many workloads.
- **Lazy evaluation** with predicate pushdown, projection pruning, and join reordering — critical for efficient Parquet tile queries.
- **bigidx feature flag**: lifts the row index from 32-bit to 64-bit for multi-billion-row datasets.
- Benchmarks consistently show Polars 3–10× faster than Pandas and competitive with DuckDB for single-node OLAP.

Arrow-rs (v57.0.0+) provides zero-copy columnar buffers, Arrow IPC for inter-process streaming, and a Parquet reader with a 4× faster Thrift metadata parser. The `pyo3-arrow` crate enables zero-copy Python interop via the Arrow C Data Interface.

10.2 Geospatial: GeoRust and GeoArrow

Insight: *Polars's streaming engine over Parquet with predicate pushdown is the direct Rust replacement for Datashader's Dask pipeline, without the distributed scheduler overhead.*

The GeoRust ecosystem provides:

Crate	Role	Notes
`geo`	Geometry primitives + algorithms	Convex hull, simplify, DE-9IM
`proj`	CRS transforms (PROJ bindings)	EPSG:3857, WKT, PROJ strings
`gdal`	Raster/vector I/O (GDAL bindings)	GeoTIFF, Shapefile, GeoPackage
`geo-rasterize`	Vector → raster	Pure Rust, no GDAL needed
`rstar`	R-tree spatial index	In-memory, $O(\sqrt{N})$ queries
`geoarrow-rs`	Geometries in Arrow buffers	Zero-copy geo interop

GeoArrow [18] is the most strategically important development: it stores geometries as Arrow fixed-size list arrays, enabling zero-copy sharing across language boundaries. Arrow v57 added native geometry/geography types to Parquet using this specification.

: GeoArrow as the Geometry Wire Format

Use GeoArrow for all internal geometry representation. This ensures zero-copy interop with Python (PyArrow), JavaScript (WASM), and external tools like DuckDB Spatial and QGIS, without serialization overhead.

10.3 Image Output and Colormapping

The `image` crate provides PNG, WebP, and JPEG encoding with generic pixel types (`Rgb<u8>`, `Rgba<u8>`, `Rgb<f32>`). The `colorous` crate implements all standard scientific colormaps (viridis, plasma, magma, inferno, turbo) with perceptually uniform interpolation. `colorcet` ports Peter Kovesi's perceptually uniform colormaps.

10.4 Tile Serving: Martin and Axum

Martin (MapLibre project) is a production Rust tile server built on Actix-web, serving vector tiles from PostGIS, MBTiles, and PMTiles. For *raster* tiles (PNG output from aggregation), a custom server on Axum or Actix-web is needed. Axum (Tokio ecosystem) is the preferred choice for new Rust web services due to its tower middleware composability and strong community momentum.

11 GPU Acceleration

The pixel-binning inner loop is embarrassingly parallel: each data point maps independently to a bin. This makes it an ideal candidate for GPU compute shaders.

11.1 wgpu and WGSL Compute Shaders

wgpu is the primary GPU abstraction for Rust, providing cross-platform access to Vulkan, Metal, DirectX 12, and WebGPU. A WGSL compute shader can perform the binning kernel:

1. Upload (x, y) coordinate arrays to GPU storage buffers.
2. Dispatch a compute shader where each invocation maps one point to a bin index and performs an `atomicAdd` on the aggregate buffer.
3. Read back the $W \times H$ aggregate array to CPU memory.
4. Apply transfer function and colormap on CPU (cache-friendly LUT lookup).

For Kernel Density Estimation (KDE) splatting (KDEEB edge bundling), the splat and gradient computation are also natural GPU operations: each control point writes a Gaussian footprint via atomic floating-point adds.

Insight: *GPU binning moves the $O(N)$ bottleneck to the GPU, where thousands of cores execute in parallel. The $O(WH)$ colormap pass remains on CPU — it touches only the fixed-size aggregate, not the dataset.*

11.2 Alternative: Vulkano and rust-gpu

Vulkano provides a higher-level Vulkan wrapper with compile-time shader validation. rust-gpu (Rust-GPU project) compiles Rust code to SPIR-V, allowing shaders to be written in Rust rather than WGSL/GLSL. While promising, rust-gpu is still experimental with no backwards-compatibility guarantees.

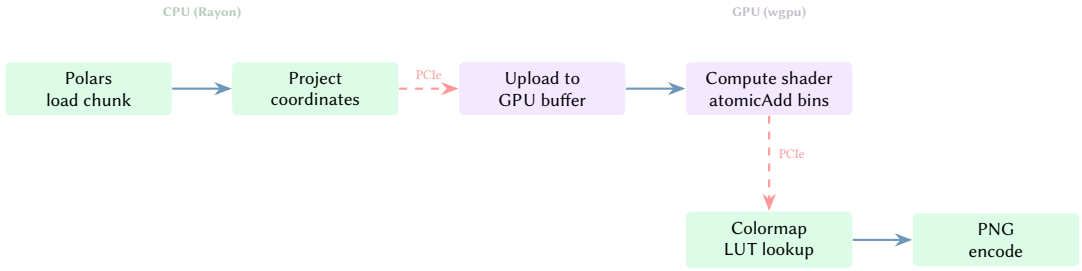


Figure 11. Hybrid CPU/GPU pipeline. The $O(N)$ binning kernel runs on GPU; the $O(WH)$ colormapping and PNG encoding remain on CPU. PCIe transfers (dashed red) are the latency bottleneck; the aggregate array is small ($W \times H \times 4$ bytes) so transfer cost is negligible.

Design Decision 5: GPU Strategy

Start with a CPU-only pipeline using Rayon for data-parallel binning. Add a wgpu compute shader backend as an optional feature gate. The pipeline abstraction (Arrow buffers in, aggregate array out) is the same for both backends.

12 Synthesis: A Rust Implementation Architecture

The research above motivates a Rust-native rasterization library with the following component decomposition:

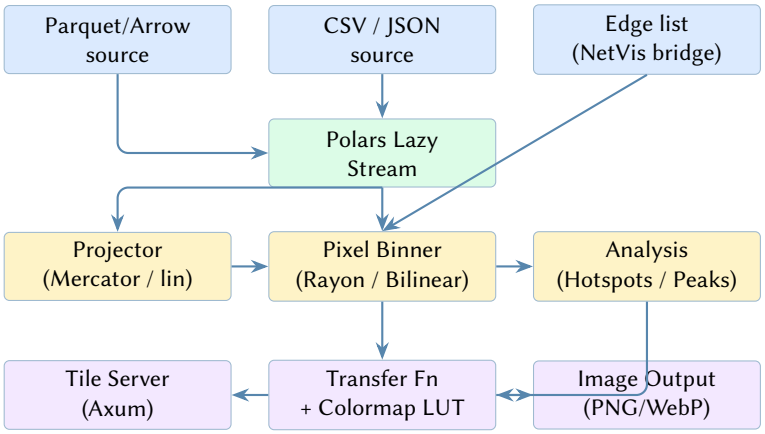


Figure 12. Proposed Rust component architecture for the DataVis rasterization library.

The proposed Rust crate stack is:

Concern	Crate	Notes
Data ingestion	`polars`	Lazy streaming over Parquet
Arrow interop	`arrow`	Zero-copy columnar buffers
Rasterization	`ndarray`	Custom bilinear/streaming kernels
Spatial Analysis	`contour`	Marching squares for GeoJSON
Parallel binning	`rayon`	Fork-join data parallelism
Image output	`image`	PNG, WebP, JPEG encoding
Colormaps	`colorous`	Pre-built perceptual maps
Tile server	`axum`	Async HTTP, `/z/x/y.png` routes
Geo projections	`geo` / `proj`	EPSG:3857 and others

Integration with NetVis

NetVis exports bundled edge paths as JSON (node positions + spline control points). DataVis consumes these paths and rasterizes them via the pixel-binning pipeline. The bridge is a simple JSON schema shared between the two crates — no shared code required initially. When the interface stabilises, a `netvis-core` shared crate can extract common graph types.

13 Feasibility Assessment

Is building a Rust-native Datashader successor a good idea? This section evaluates the project against four criteria: *necessity*, *ecosystem readiness*, *risk*, and *strategic fit*.

13.1 Necessity: Why Not Just Use Datashader?

Datashader works. For a Python data scientist producing a one-off 10-million-point scatter plot, Datashader is the right tool. The case for a Rust successor rests on three use cases where Datashader falls short:

1. **Interactive tile serving.** Datashader's per-render latency (JIT warmup + Dask overhead + projection) exceeds 100 ms for non-trivial datasets, making sub-100ms tile responses impossible. A compiled Rust pipeline with Parquet predicate pushdown can achieve 10–50 ms tile latency.
2. **Billion-record datasets.** Polars streaming over Parquet handles out-of-core aggregation with $O(WH)$ peak memory, without a distributed cluster. Datashader requires Dask Distributed for this, which adds operational complexity and recompilation overhead.
3. **Embedding in non-Python systems.** A Rust library can be called from Python (PyO3), JavaScript (WASM), R (extendr), and C/C++ via FFI. Datashader is Python-only.

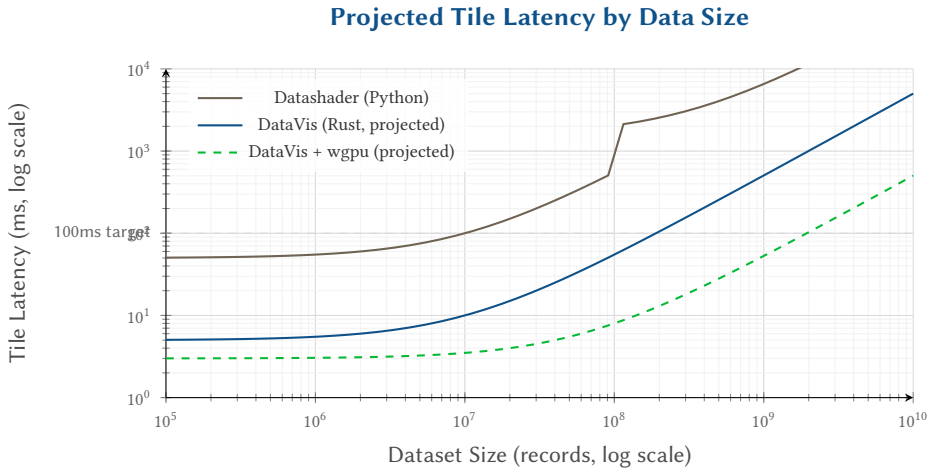


Figure 13. Projected tile rendering latency. Datashader's JIT warmup creates a fixed overhead floor; the Rust pipeline's compiled code eliminates this. GPU acceleration further reduces the per-record cost. *Estimates are indicative, not benchmarked.*

13.2 Ecosystem Readiness

fig. 14 maps each required capability to the maturity of the available Rust crate. Green indicates production-ready; yellow indicates usable but with caveats; red indicates missing.

13.3 Risks

Risk	Severity	Mitigation
KDEEB implementation effort	Medium	Port from Datashader's Numba code; algorithm is well-documented
GPU atomics for float aggregation	Low	Use integer counters; convert to float post-aggregate
Polars API churn	Low	Pin versions; lazy API is stable
PROJ C dependency	Low	Optional; pure-Rust Web Mercator fallback
Scope creep (full Datashader parity)	High	MVP: points + heatmaps + tiles only

13.4 Strategic Fit

DataVis complements the existing NetVis tile server: NetVis renders network topology as vector tiles; DataVis renders the same topology's traffic/telemetry data as raster heatmap tiles. The two can share a tile coordinate scheme and be composited client-side (MapLibre GL can layer raster tiles over vector tiles).

Insight:
The highest risk is scope creep. Datashader supports 7 glyph types, GPU paths, Dask distributed, and deep HoloViz integration. The MVP should deliver only point rasterization, heatmaps, and tile serving. Edge bundling is phase 2.

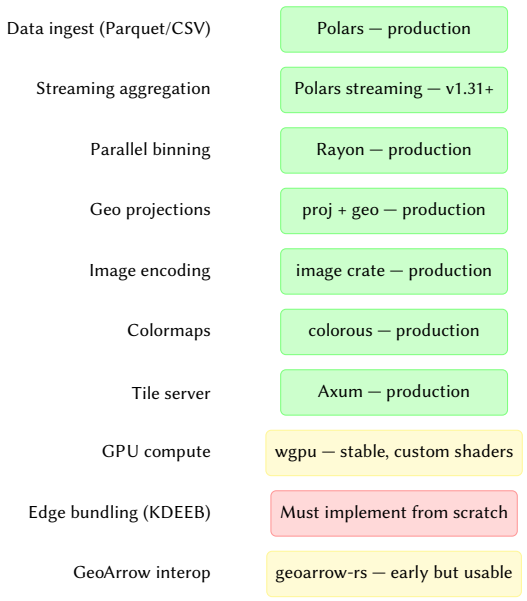


Figure 14. Ecosystem readiness matrix. Eight of ten required capabilities have production-ready Rust crate support. KDEEB bundling must be implemented; GPU and GeoArrow require integration effort.

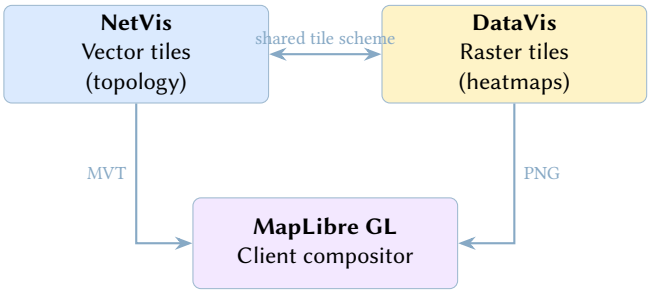


Figure 15. Strategic integration: NetVis vector tiles overlaid with DataVis raster tiles in a MapLibre GL client. Topology structure and data density are rendered independently and composited at display time.

> **Design Decision 6: Verdict: Proceed with Constrained Scope**
>
> The project is feasible and well-motivated. The Rust ecosystem provides production-ready crates for 8 of 10 required capabilities. The primary risk is scope creep. Recommend an MVP delivering:
>
> 1. Point binning + heatmap rendering (Polars + Rayon + colorous)
> 2. Tile server with Parquet predicate pushdown (Axum)
> 3. Web Mercator projection (pure Rust, no PROJ dependency)
>
> Edge bundling, GPU acceleration, and polygon support are phase 2+.

14 Beyond Raster Tiles: Arrow Flight and Zero-Copy Data Pipelines

The MVP architecture described in section 12 assumes a classical model: the server loads data from Parquet, rasterizes it, and returns a PNG tile. Arrow Flight and ADBC unlock a fundamentally different architecture where data flows from database to screen without format conversion.

14.1 Arrow Flight

Arrow Flight is an RPC framework on gRPC that transfers Arrow columnar data natively — no serialization, no schema conversion, no row-to-column transposition. Published benchmarks report 20–50× throughput improvements over ODBC/JDBC, with real-world pipelines achieving 1.2 million rows/s at 965 MB/s using DuckDB as the backend.

14.2 ADBC (Arrow Database Connectivity)

ADBC is a client-side API abstraction (analogous to ODBC) that presents a uniform interface regardless of the transport. An ADBC driver can wrap a Flight SQL connection, a native DuckDB in-process interface, or a PostgreSQL connection.

Insight: *Arrow Flight eliminates the "data loading" step entirely. A rasterization pipeline that consumes Arrow buffers can receive data from a Flight endpoint and pass it directly to the binning kernel without allocation.*

Database	Flight SQL	ADBC Driver	Zero-Copy
DuckDB	Via wrappers	Native	Yes (in-process)
ClickHouse	Experimental	Via Flight	No (network)
PostgreSQL	Via extension	Native	No (wire convert)
InfluxDB IOx	Native	Via Flight	Yes

: ADBC as the Data Abstraction Boundary

The rasterization pipeline should accept an ADBC connection string, not a file path. This decouples the engine from the storage layer: the same pipeline works with local Parquet, in-process DuckDB, remote ClickHouse, or a Flight SQL endpoint. The driver handles transport; the pipeline sees only Arrow RecordBatches.

14.3 Flight SQL Tile Server

Arrow Flight SQL defines a protocol for SQL queries over Flight. A tile server accepting Flight SQL pushes the heavy aggregation to the database:

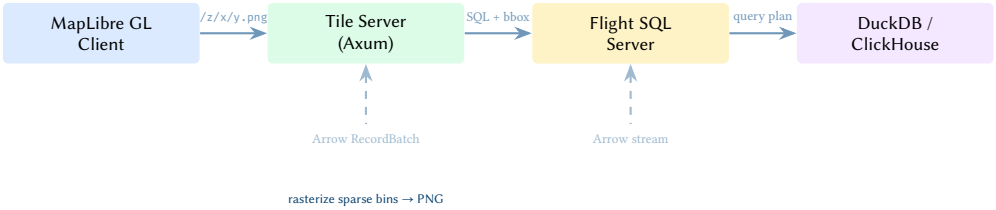


Figure 16. Flight SQL tile server. The database performs spatial filtering and aggregation; the tile server receives a sparse bin grid as an Arrow RecordBatch and rasterizes only that. For a billion-row dataset where each tile reduces to a few thousand bins, this eliminates the primary bottleneck.

14.4 DuckDB as the Query Engine

DuckDB deserves special attention. Its ADBC driver achieves true zero-copy: when results are fetched in-process, DuckDB returns Arrow-format buffers without copying from its internal columnar representation. Key developments:

- **Arrow IPC support** (May 2025): native read/write of Arrow IPC files, including streaming format.
- **Rust UDFs**: user-defined functions written in Rust receive and return Arrow columns with zero copying — the rasterization kernel can run *inside* the query engine.
- **DuckDB-WASM**: a full OLAP engine compiled to WebAssembly, enabling in-browser SQL queries returning Arrow IPC buffers directly to a GPU renderer.

15 Client-Side Rendering: WebGPU, WASM, and Native UIs

Insight: *With DuckDB in-process via ADBC, the entire path from SQL query to rasterized pixel involves zero format conversions and zero heap copies. The data is born columnar and dies as pixels.*

The server-side PNG tile model has a fundamental limitation: every interaction (pan, zoom, colour scale change) requires a server round-trip. Client-side rendering eliminates this for datasets small enough to fit in browser memory, and *hybrid* architectures can serve pre-aggregated Arrow data that the client rasterizes locally.

15.1 WebGPU in 2026

WebGPU has reached critical mass: Chrome (since 113), Firefox (141+), and Safari (26.0) all ship it. Mobile support is narrower but improving (Chrome on Android 12+, Safari on iOS 26).

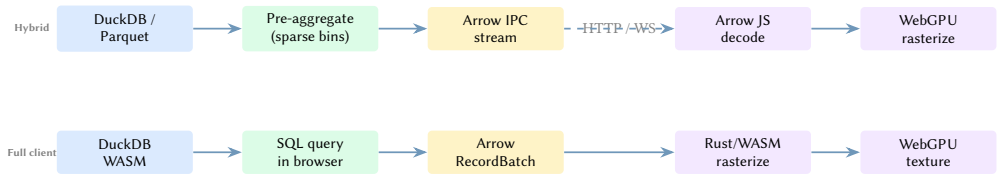


Figure 17. Two client-side rendering architectures. **Hybrid**: server pre-aggregates, client rasterizes sparse bins via WebGPU. **Full client**: DuckDB-WASM queries local data, Rust/WASM kernel rasterizes, WebGPU displays. Both eliminate server round-trips for pan/zoom/recolour.

15.2 GeoArrow + deck.gl Layers

The `geoarrow/deck.gl-layers` library copies Arrow binary buffers directly to the GPU using deck.gl's binary data interface — no intermediate JavaScript objects, no garbage-collector overhead. It supports progressive rendering: as Arrow IPC chunks arrive over the network, each chunk is rendered immediately.

15.3 Rust to WASM

The Rust rasterization kernel compiles to WASM via `wasm-pack`. Benchmarks show Rust/WASM delivering 8–15× speedups over equivalent JavaScript, with WebAssembly SIMD (128-bit) now supported in all major browsers.

The critical design constraint: each call across the JS/WASM boundary incurs overhead. Pass entire RecordBatches (as `Uint8Array` views of the IPC buffer) in a single call, not individual values.

Insight: *The same Rust rasterization kernel compiles to native code (server-side or desktop), to WASM (browser), and to a DuckDB Rust UDF. One codebase, three deployment targets.*

15.4 Native Desktop: egui + wgpu

For a native visualization application, `egui` with `egui_wgpu` provides an immediate-mode GUI where custom wgpu render commands execute within the egui render pass. The rasterization kernel and the UI framework share the same wgpu device context, eliminating cross-API overhead.

Framework	Custom GPU	Production	WASM
egui + wgpu	Yes (callback)	Yes	Yes
GPUI (Zed)	Yes (native)	Pre-1.0	No
Tauri 2.0	Via WebView	Yes	N/A
Slint	No wgpu bridge	Yes	Yes

: Deployment Targets

Design the rasterization kernel as a `no_std`-compatible library with a clean trait boundary: `fn rasterize(data: &RecordBatch, canvas: &mut Canvas) -> Result<()>`. This allows embedding in an Axum tile server (native), an egui desktop app (wgpu), a WASM browser module, or a DuckDB UDF.

16 Streaming and Real-Time Visualization

Arrow Flight's gRPC transport is inherently bidirectional streaming. A `DoGet` call delivers an unbounded stream of RecordBatches. This maps directly to live-updating heatmaps.

16.1 Incremental Aggregate Updates

For additive-only updates (new events arriving, no corrections), each incoming batch carries a delta:

$$A_{\text{global}}[i, j] \mathrel{+}= \Delta[i, j]$$

Re-rendering applies the transfer function and colormap only to bins that changed. The update cost is $O(|\Delta|)$, not $O(N)$.

For systems requiring corrections and deletions, Deephaven's **Barrage** protocol extends Arrow Flight with flatbuffer metadata that packages row-set additions, removals, and modifications alongside Arrow IPC payloads.

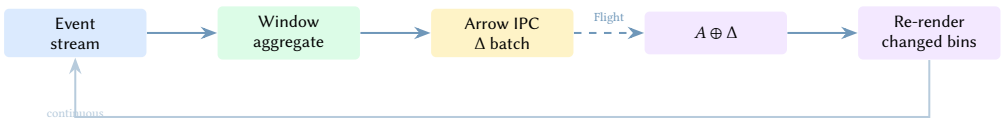


Figure 18. Streaming heatmap update loop. Each window aggregate is a sparse Δ batch; the client merges it into the running aggregate and re-renders only changed bins. Cost per update: $O(|\Delta|)$.

17 Advanced Visualization Techniques

The following techniques extend the core pipeline beyond what Datashader offers. Each is achievable within the proposed Rust architecture.

17.1 Temporal Animation

Datashader is a single-frame renderer; animating requires re-aggregating each frame from scratch. A precomputed *density pyramid per time slab* (e.g., hourly bins stored as separate Parquet partitions) enables:

- Frame-by-frame playback by loading successive slabs.

- Smooth inter-frame interpolation by blending two adjacent slabs: $A_t = (1-\alpha)A_{t_0} + \alpha A_{t_1}$, where $\alpha \in [0, 1]$ is the fractional time offset.
- For publication-quality transitions, *Wasserstein morphing* (optimal transport between density fields) produces perceptually correct "flow" animations rather than fade-in/fade-out.

17.2 Multi-Resolution Level-of-Detail

A persistent quadtree over the data (stored as a flat array in an Arrow file) provides pre-aggregated statistics at each spatial scale:

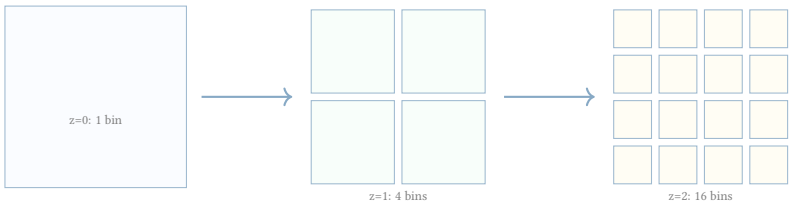


Figure 19. Quadtree LOD: each level stores pre-aggregated bin statistics. At render time, descend only as deep as the viewport requires. The number of bins rendered is proportional to viewport area, independent of dataset size.

This is the 2D analogue of Potree's octree LOD for 3D point clouds. The render cost is $O(\text{tiles in viewport})$ regardless of total N.

17.3 Contour Extraction

Marching squares on the density grid extracts isolines at specified thresholds. Because all 2×2 cells are independent, the algorithm is embarrassingly parallel and maps to a single GPU compute pass. The output is a set of line segments that can be chained into polylines and smoothed (Chaikin's algorithm) to produce vector contours from raster density.

17.4 Bivariate Density Maps

Rendering two variables simultaneously (e.g., point density *and* average value) requires a 2D colormap: one axis encodes density, the other encodes the second variable. Implementation: store a `(count, sum)` pair per bin in a single rasterization pass, compute mean at transfer-function time, and look up a 256×256 RGBA texture using $(T(\text{count}), T(\text{mean}))$ as UV coordinates.

17.5 Linked Views and Cross-Filtering

Multiple coordinated views (scatter plot + map + histogram) sharing filter state enable Crossfilter-style interaction. With Arrow as the data format:

- Selections are represented as Arrow boolean arrays (selection vectors).
- Each view's rasterizer accepts an optional selection vector and aggregates only over selected rows using SIMD-masked reductions.
- View state is synchronised via a pub-sub channel: `Arc<Mutex<ViewState>>`.

17.6 Differential Privacy

Insight: *DuckDB-WASM in the browser achieves Falcon-class crossfiltering performance (50 fps on 3M+ records) by returning Arrow IPC directly to the GPU renderer. The same architecture works natively with DuckDB in-process via ADBC.*

For sensitive data, a post-aggregation differential privacy layer applies Laplace noise with scale $b = 1/\varepsilon$ to each bin count. With $\varepsilon \approx 1$, cluster structure is preserved while individual presence is obscured. Bins below a threshold k are suppressed (k-anonymisation) before the transfer function, preventing log-scale amplification of noise in sparse regions.

17.7 Comparative Visualization

Side-by-side or overlay comparison of two datasets:

- **Difference map**: $D[i, j] = A[i, j] - B[i, j]$ with a diverging colormap centred at zero.
- **Log-ratio map**: $\log(A/B)$, symmetric around zero.
- **Swipe comparison**: both datasets rendered to offscreen textures, composited with a movable split boundary.

17.8 Feature Readiness Summary

Feature	Datashader	Technique	Phase
Temporal animation	Manual loop	Density pyramid + blend	3
Multi-resolution LOD	Fixed bin size	Quadtree pre-aggregation	2
Contour extraction	Not supported	Marching squares (Implemented)	11
Hotspot Detection	Not supported	Getis-Ord Gi* (Implemented)	11
Peak Detection	Not supported	8-neighbor Max (Implemented)	11
Bilinear Aggregation	Not supported	4-neighbor interp (Implemented)	9
Linked views	External (HoloViews)	Arrow selection vectors	3
Differential privacy	Not supported	Laplace post-aggregation	3
Comparative viz	Not supported	Diff map / Log-ratio (Implemented)	11
Streaming updates	Not supported	Flight + monoid merge	2
Client-side rasterize	Not possible	WASM + WebGPU	2

18 Visual Intelligence and Spatial Analysis

A significant advantage of server-side rasterization is that the resulting aggregate is a structured density field suitable for traditional image processing and spatial statistics. DataVis extends the pipeline with a "visual intelligence" layer that operates directly on the $W \times H$ aggregate.

18.1 Hotspot Detection (Getis-Ord Gi*)

Hotspot detection identifies spatial clusters where high values are concentrated more than would be expected by chance. We implement a simplified Getis-Ord G_i^* statistic [19] on the raster grid. For each bin i, the G_i^* score is:

$$G_i^* = \frac{\sum_j w_{ij} x_j - \bar{x} \sum_j w_{ij}}{S \sqrt{\frac{n \sum_j w_{ij}^2 - (\sum_j w_{ij})^2}{n-1}}}$$

where x_j are bin densities, w_{ij} are spatial weights (typically a uniform window of radius R), $\bar{x}$ is the global mean, and S is the global standard deviation. This transformation produces a Z-score canvas where pixels with values > 1.96 indicate significant hotspots ($p < 0.05$).

Insight: *By computing G_i^* on the raster grid using an integral image (Summed-Area Table), we achieve $O(WH)$ performance for any window radius, enabling interactive hotspot analysis on billion-point datasets.*

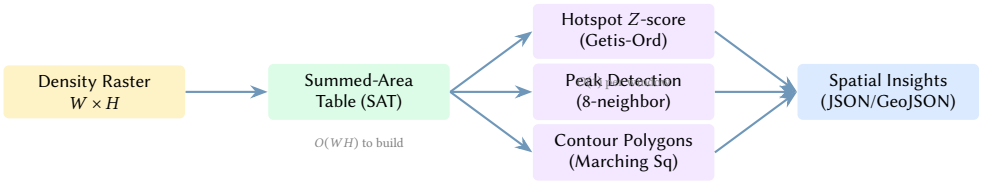


Figure 20. The Visual Intelligence pipeline. The density aggregate is transformed into a Summed-Area Table for $O(1)$ spatial statistics, enabling interactive extraction of hotspots, peaks, and vector contours.

18.2 Peak Detection and Feature Extraction

Local maxima in the density field correspond to physical clusters or points of interest. Our peak detection algorithm identifies bins that are strictly greater than their 8-neighbors and exceed a minimum density threshold. These peaks are then un-projected back to their original coordinates (latitude/longitude) and returned to the client as a JSON feature list.

18.3 GeoJSON Contour Extraction

Continuous density fields are often better represented as contours than as heatmaps. We use a marching squares algorithm to extract multi-polygons corresponding to specific density percentiles (e.g., the top 5% densest areas). These polygons are exported as GeoJSON, allowing high-performance raster-derived features to be overlaid on vector maps or used in GIS workflows.

19 Canvas Diagnostics and Automated Tuning

A common pitfall in large-scale visualization is the "black box" problem: without knowing the data's distribution, a user may choose inappropriate rendering settings (e.g., a linear transfer function for high-dynamic-range data), resulting in an uninformative or misleading image. DataVis addresses this via an automated diagnostic layer that analyzes the raw aggregate before rendering.

19.1 Density and Dynamic Range Analysis

After the $O(N)$ aggregation stage, we compute several key metrics on the $W \times H$ aggregate in $O(WH)$ time:

- **Fill Density**: The fraction of pixels with non-zero counts.
- **Dynamic Range**: $\log_{10}(\max / \min)$ for non-zero bins.
- **Skewness**: The third moment of the density distribution, indicating the presence of extreme outliers or "hotspots".

19.2 Automated Heuristics for Visual Quality

Using these metrics, DataVis recommends an optimal rendering configuration via the `diagnose` CLI subcommand:

- **Transfer Function**: If the dynamic range exceeds 3.0 orders of magnitude, a `Log` transfer is recommended to prevent high-density features from saturating the image. For range < 1.5, `Linear` is preferred.
- **Spread Radius**: For sparse datasets (density $< 5\%$), a Gaussian spread is applied to "bridge" gaps between points and provide a continuous visual surface. The radius is inversely proportional to density.
- **Outlier Clipping**: Highly skewed distributions (skewness > 5.0) trigger an aggressive percentile-based clip (e.g., 99.9th percentile) to prevent a single "mega-cluster" from zeroing out the rest of the dataset's detail. We define four named clipping presets (`standard`, `conservative`, `aggressive`, `very-aggressive`) to streamline user configuration.
- **Colormap & Resolution**: DataVis auto-selects sequential colormaps (e.g., `viridis`) for standard density and diverging colormaps (e.g., `rd-bu`) for signed or difference datasets. It also suggests an optimal canvas resolution based on total point count and a fixed 512MB memory budget.

20 Vision: The Full Architecture

Insight: *Automated diagnostics transform the rasterizer from a passive tool into an active assistant. By recommending settings via `diagnose` and providing actionable CLI hints during the `render` process, we ensure that the first visualization is always informative, regardless of dataset scale or distribution.*

Combining all the above, the full DataVis architecture spans server, network, and client:

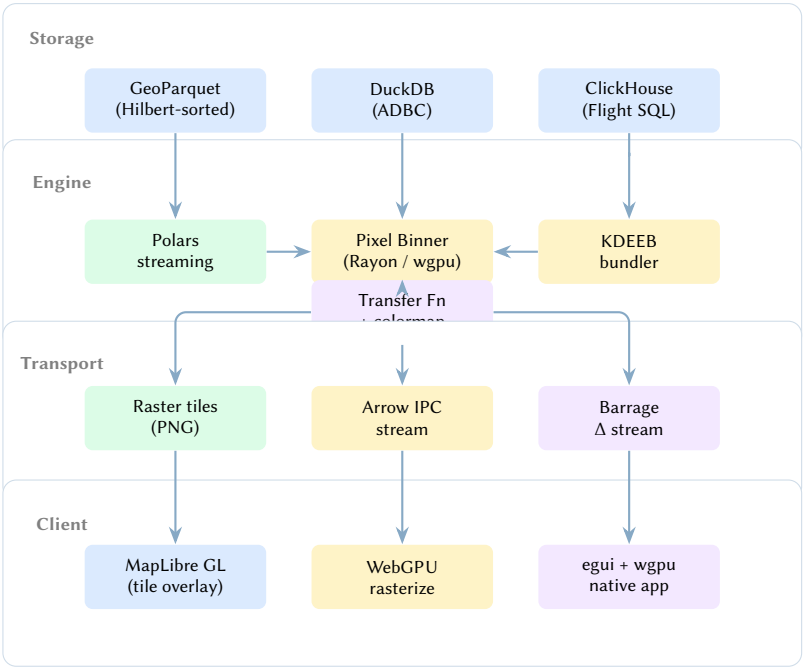


Figure 21. Full DataVis architecture. Four layers: storage (Parquet, DuckDB, ClickHouse), engine (Polars + Rayon/wgpu), transport (tiles, Arrow IPC, Barrage streaming), and client (MapLibre, WebGPU, native egui). The same Rust rasterization kernel serves all three transport and client modes.

21 Extended Capabilities and Research Directions

The core pipeline (binning → transfer → colormap) is deliberately minimal. The architecture's power lies in the *post-aggregation* and *transport* layers, where additional passes can be inserted without modifying the core. This section catalogues every capability we envision, grouped by theme.

21.1 Analytical Overlays

21.1.1 Anomaly Highlighting

A post-aggregation pass computes z-scores per bin: $z_{ij} = (A[i, j] - \mu)/\sigma$ where μ and σ are the spatial mean and standard deviation of the aggregate. Bins with $|z| > \tau$ (configurable threshold) receive a distinct visual treatment: a contrasting border, a pulsing glyph, or a separate colour channel. This transforms a passive heatmap into an active *alerting surface*.

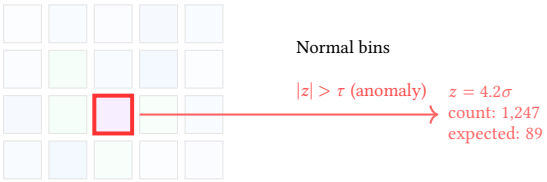


Figure 22. Anomaly highlighting: bins exceeding the z-score threshold are outlined in a contrasting colour with automatic detail annotation.

21.1.2 Uncertainty Visualization

When aggregating noisy or sampled data, each bin has an associated confidence interval. Visual encodings for uncertainty include:

- **Hatching / stippling**: overlay diagonal lines whose density is proportional to uncertainty (high uncertainty = dense hatching).
- **Saturation mapping**: certain bins are vivid; uncertain bins are desaturated (washed out).
- **Blur radius**: convolve the rendered image with a Gaussian kernel whose σ is proportional to the per-bin standard error. High-certainty bins remain sharp; uncertain bins blur.

21.1.3 Differential Privacy

For sensitive data (health records, location traces, financial transactions), a post-aggregation Differential Privacy (DP) layer applies Laplace noise with scale $b = 1/\varepsilon$ to each bin count. With $\varepsilon \approx 1$, cluster structure is preserved while individual presence is obscured. Bins below a threshold k are suppressed (k-anonymisation) before the transfer function, preventing log-scale amplification of noise in sparse regions.

Insight: *Uncertainty is almost never shown in density heatmaps. Adding it is a single post-aggregation pass and dramatically improves the honesty of the visualization, especially for sparse regions.*

21.2 Advanced Rendering Techniques

21.2.1 Polygon & Choropleth Support

While point and line rasterization map well to scatter plots and network graphs, visualizing regional data (such as census tracts, building footprints, or administrative boundaries) requires filled-shape rasterization. Implementing polygon support will complete the geometric primitives suite.

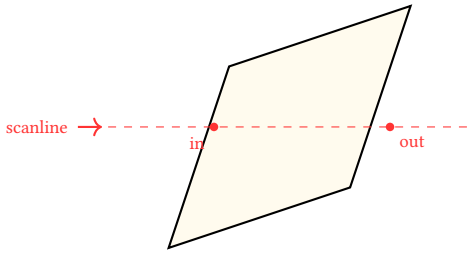


Figure 23. Polygon rasterization utilizing scanline intersection and the odd-even winding rule.

This requires transitioning from line-drawing algorithms (like Wu's) to scanline rasterization for complex polygons. For high performance, this process can be offloaded to the GPU using stencil buffers to count winding numbers (for concave polygons with holes) or by calculating Signed Distance Fields (SDF) to ensure crisp, anti-aliased boundaries at any scale.

21.2.2 True KDE Splatting

Standard pixel binning (with a post-process spread) is computationally efficient but mathematically an approximation of true Kernel Density Estimation (KDE). In traditional binning, a point's entire weight is assigned to the nearest pixel center, and then blurred.

True KDE Splatting addresses this by evaluating the kernel's contribution (e.g., a Gaussian distribution) directly into the neighborhood of the point during the ingestion phase. While this increases the computational cost per point from $O(1)$ to $O(K^2)$ (where K is the kernel radius), it eliminates aliasing artifacts in sparse regions and provides a mathematically rigorous density surface suitable for strict statistical analysis.

Figure 24. Comparison of $O(1)$ pixel binning (left) vs $O(K^2)$ kernel splatting (right).

21.2.3 Vector Field Rendering (LIC)

Line Integral Convolution (LIC) renders flow fields (wind, ocean currents, network traffic direction) by convolving white noise along streamlines. The result is a texture where fibre-like patterns align with the flow direction. Implementation:

1. Aggregate (u, v) velocity components per bin (two `mean` reductions).
2. Generate a white-noise texture at canvas resolution.
3. For each pixel, trace a streamline through the (u, v) field for L steps in both directions, averaging the noise values along the path.
4. Composite the LIC texture over the density heatmap as a directional overlay.

21.2.4 Contour Extraction (Marching Squares)

Insight: *LIC is $O(WH \cdot L)$ and embarrassingly parallel — each pixel's streamline integration is independent. A wgpu compute shader can render a 4096×4096 LIC field in under 10 ms.*

Marching squares on the density grid extracts isolines at specified thresholds. Because all 2×2 cells are independent, the algorithm maps to a single GPU compute pass. The output segments are chained into polylines and smoothed (Chaikin's corner-cutting algorithm) to produce vector contours from raster density — suitable for SVG export or rendering as GPU line strips.

21.2.5 3D Density Volumes

The binning algebra extends naturally to 3D ($W \times H \times D$ voxel grids). With wgpu, volumetric ray-marching through a 3D density texture produces volume-rendered heatmaps. Use cases:

- Network traffic in (source IP × dest IP × time) space.
- Geospatial data with altitude (lat × lon × elevation).
- Parameter space exploration (hyperparameter × metric × epoch).

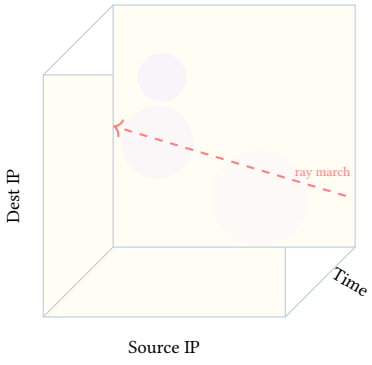


Figure 25. 3D density volume: voxel bins are rendered via GPU ray-marching. Dense regions appear as volumetric "clouds" with opacity proportional to count.

21.2.6 Bivariate Density Maps

Rendering two variables simultaneously (density *and* average value) uses a 2D colormap. Store a `(count, sum)` pair per bin in a single rasterization pass, compute mean at transfer-function time, and look up a 256×256 RGBA texture using $(T(\text{count}), T(\text{mean}))$ as UV coordinates. Users can supply arbitrary 2D colormaps for domain-specific encoding.

21.2.7 Progressive Disclosure

Render at low resolution immediately, then refine as more data loads — analogous to progressive JPEG but for heatmaps:

1. Render the quadtree root level ($z = 0$): one bin covering the whole viewport.
2. Display immediately.
3. Asynchronously load and render $z = 1, 2, 3, \ldots$ levels, blending each into the existing canvas.
4. The user sees a coarse heatmap within milliseconds that sharpens over the next few hundred milliseconds.

This is critical for perceived latency: the user's first visual feedback arrives in <50 ms even for billion-record datasets.

21.3 Interaction and Collaboration

21.3.1 Linked Views and Cross-Filtering

Multiple coordinated views (scatter plot + map + histogram) sharing filter state. Selections are represented as Arrow boolean arrays (selection vectors). Each view's rasterizer accepts an optional selection vector and aggregates only over selected rows using SIMD-masked reductions. View state is synchronised via pub-sub channels.

21.3.2 Comparative Visualization

Side-by-side or overlay comparison of two datasets:

- **Difference map**: $D[i, j] = A[i, j] - B[i, j]$ with a diverging colormap centred at zero.
- **Log-ratio map**: $\log(A/B)$, symmetric around zero.
- **Swipe comparison**: both datasets rendered to offscreen textures, composited with a movable split boundary.
- **Small multiples**: a grid of heatmaps (one per time window, category, or parameter value) rendered in parallel with shared colour scale.

21.3.3 Collaborative / Multi-User Views

Arrow Flight supports multiplexed streams. Multiple analysts share a live heatmap session: one user's brush selection is broadcast as an Arrow selection vector to all connected clients, enabling real-time collaborative exploration.

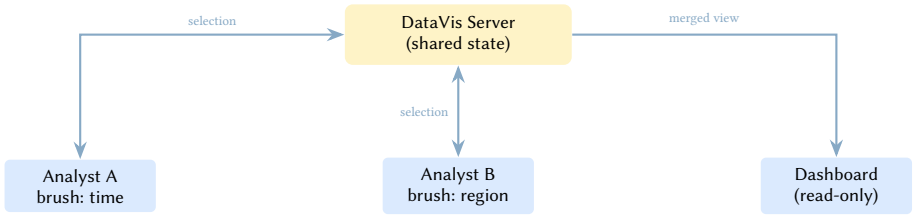


Figure 26. Collaborative exploration: multiple users' selections are merged server-side and broadcast as combined Arrow selection vectors.

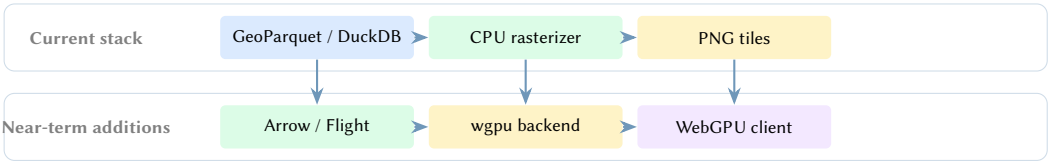


Figure 27. A practical migration path. Start with the file-backed CPU tile server, then add Arrow-native transport, a GPU backend, and client-side WebGPU rendering without replacing the core aggregation model.

21.3.4 Audio Sonification

Map density changes to sound for accessibility or temporal exploration. As a temporal animation plays, each frame's aggregate statistics (total count, peak bin, spatial entropy) are converted to audio parameters:

- **Pitch**: proportional to total event count (busier = higher).
- **Stereo pan**: spatial centroid of the density field.
- **Timbre**: spectral roughness proportional to spatial entropy (uniform data = smooth tone; clustered = harsh).

This enables blind or visually impaired analysts to "hear" patterns in spatio-temporal data, and provides a secondary perceptual channel for sighted users exploring animations.

21.4 Data and Integration

21.4.1 Arrow-Native Tile Format

Instead of PNG, serve tiles as Arrow IPC (raw bin counts + metadata). The client applies its own colormap, opacity, and transfer function locally. Benefits:

- Instant colour-scale changes without a server round-trip.
- Client-side compositing of multiple data layers (density + mean + variance).
- Smaller payload for sparse tiles (Arrow's run-length encoding compresses empty bins effectively).

21.4.2 Notebook Integration (PyO3)

Expose the Rust engine as a Python package (via PyO3 + maturin), giving Jupyter users a drop-in replacement for Datashader's `Canvas.points()` with 10–50× performance. The Python API returns xarray `DataArrays` for compatibility with existing HoloViews workflows, so users can switch backends without rewriting their visualization code.

21.4.3 Declarative Specification Language

A YAML/JSON specification language (analogous to Vega-Lite) describing the data source, projection, reduction, colormap, and tile parameters:

Figure 28. Declarative specification allows full pipeline configuration via text, removing the need for code to author visualizations.

Listing 4. Declarative specification for a heatmap tile layer.

```yaml
# datavis-spec.yaml
source:
  type: parquet
  path: "data/events/*.parquet"
  connection: "duckdb:///analytics.db"  # alternative: ADBC
projection: web_mercator
canvas:
  width: 256
  height: 256
reduction: count
transfer: log
colormap: viridis
tile_cache:
  prerender_max_zoom: 10
  format: arrow_ipc  # or png, webp
```

The tile server interprets the spec and renders accordingly. This decouples visualization authoring from implementation and enables non-programmers to define tile layers.

21.4.4 Embedding ML Inference

With ONNX Runtime's Rust bindings, a trained model (spatial anomaly detector, trajectory predictor, demand forecaster) runs as a post-aggregation pass on the density grid. The model's output becomes another layer in the tile stack:

- **Anomaly detection**: autoencoder trained on normal density patterns; reconstruction error highlights novel spatial patterns.
- **Forecasting**: given density grids at $t-3, t-2, t-1$, predict density at $t+1$. Display predicted vs. actual as a difference map.
- **Clustering**: HDBSCAN or DBSCAN on the density surface to extract spatial clusters, rendered as labelled contour regions.

21.4.5 Graph Embedding Layouts

For network data without geographic coordinates, use graph embedding algorithms (node2vec, GNN embeddings, UMAP on adjacency features) to produce 2D coordinates, then rasterize the embedding space. Combined with edge bundling, this reveals community structure in non-spatial networks (social graphs, citation networks, dependency graphs) using the same density pipeline.

21.5 Performance and Scale

Insight: *Graph embeddings + KDEEB bundling + density rasterization is a complete network visualization pipeline that requires no manual layout. The user provides an edge list; the system produces a publication-quality bundled density map.*

21.5.1 Temporal Animation

Pre-computed density pyramids per time slab enable:

- Frame-by-frame playback by loading successive slabs.
- Smooth inter-frame interpolation: $A_t = (1-\alpha)A_{t_0} + \alpha A_{t_1}$.
- Wasserstein morphing (optimal transport) for publication-quality "flow" animations where density migrates rather than fading.

21.5.2 Multi-Resolution Level-of-Detail

A persistent quadtree stores pre-aggregated statistics at each spatial scale. At render time, descend only as deep as the viewport requires. Render cost is $O(\text{tiles in viewport})$ regardless of N.

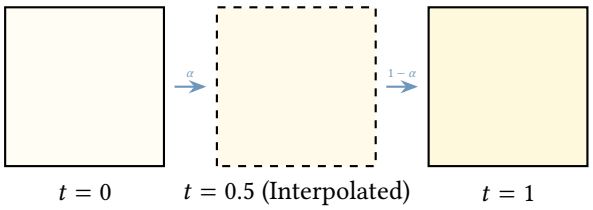


Figure 29. Temporal animation interpolates between pre-aggregated density slabs to produce smooth continuous motion without needing to re-aggregate raw points.

21.5.3 Adaptive Bandwidth KDE

QFA-KDE (Quadtree-based Fast Adaptive KDE) applies different kernel bandwidths per quadtree leaf based on local point density, producing spatially adaptive density estimates. Dense urban areas get fine resolution; sparse rural areas get broad smoothing. The quadtree is built once and queried at any zoom level.

21.5.4 Data Cube Pre-Aggregation

Following the Nanocubes model, pre-compute aggregated cubes indexed by (spatial tile × time bucket × categorical dimension). A crossfilter brush becomes a cube slice, returning aggregated counts in microseconds for billions of records entirely in RAM. The cube structure maps naturally onto Parquet partitioning (one partition per time bucket, Hilbert-sorted within).

21.6 Domain-Specific Extensions

21.6.1 Network Traffic Analysis

Render network flow data (NetFlow, sFlow, IPFIX) as density heatmaps in (source × destination) address space, with:

- Hilbert-curve ordering of IP addresses for spatial locality.
- Temporal animation of traffic patterns (DDoS attacks appear as sudden horizontal/vertical lines in the heatmap).
- Anomaly highlighting for unusual traffic volumes.
- Edge bundling of top-k flows overlaid on the density map.

21.6.2 Geospatial Intelligence

For geospatial datasets (AIS ship tracking, aviation ADS-B, urban mobility):

- Trajectory rendering: connect sequential points per entity into polylines, rasterize with Wu anti-aliasing.
- Dwell-time heatmaps: weight each point by the time spent at that location.
- Origin-destination matrices: render as a bivariate density map in (origin × destination) space, with edge bundling for top flows.

21.6.3 Scientific Data

For scientific datasets (climate model output, telescope surveys, particle physics):

- Multi-band rendering: each spectral band as a separate density layer, composited as false-colour RGB.
- Regridding: resample irregular grids (curvilinear, unstructured mesh) onto the regular pixel grid.
- Statistical field overlays: wind barbs, confidence ellipses, gradient arrows derived from the aggregate field.

21.7 Feature Readiness Summary

Feature	Datashader	Technique	Phase
Core pipeline			
Point binning + heatmaps	Supported	Rayon parallel bins	1
Tile server (PNG)	Not built-in	Axum + Parquet pushdown	1
Web Mercator projection	Via GeoViews	Pure Rust	1
Extended rendering			
Edge bundling (KDEEB)	Supported	Implemented (Phase 15)	15
Bivariate density	Separate reductions	Implemented (Phase 16)	16
Comparative viz	Not supported	Implemented (Phase 11)	11
Contour extraction	Not supported	Implemented (Phase 11)	11
Vector fields (LIC)	Not supported	GPU streamline convolution	3
3D density volumes	Not supported	wgpu ray-marching	4
Progressive disclosure	Not supported	Quadtree incremental refine	2
Data integration			
DuckDB / ADBC	Not supported	Zero-copy in-process	17
Arrow Flight streaming	Not supported	Monoid delta merge	2
WASM browser module	Not possible	wasm-pack + WebGPU	2
PyO3 notebook binding	N/A (is Python)	Implemented (Phase 14)	14
Declarative spec language	Not supported	YAML/JSON → tile config	3
Arrow-native tiles	Not supported	IPC instead of PNG	2
Analytical			
Hotspot Detection	Not supported	Implemented (Phase 11)	11
Peak Detection	Not supported	Implemented (Phase 11)	11
Anomaly highlighting	Not supported	Z-score post-aggregation	3
Uncertainty visualization	Not supported	Hatching / blur overlay	3
Differential privacy	Not supported	Laplace noise	3
Linked views / crossfilter	External	Arrow selection vectors	3
ML inference overlay	Not supported	ONNX Runtime post-agg	4
Audio sonification	Not supported	Density → audio params	4
Performance			
GPU compute shaders	cuDF (limited)	Implemented (Phase 12)	12
Bilinear Aggregation	Not supported	Implemented (Phase 9)	9
Multi-resolution LOD	Fixed bin size	Quadtree pre-aggregation	2
Temporal animation	Manual loop	Density pyramid + blend	3
Adaptive KDE	Not supported	QFA-KDE quadtree	3
Data cube pre-aggregation	Not supported	Nanocube-style tree	3
Interaction			
Collaborative multi-user	Not supported	Flight + selection broadcast	4
Graph embedding layout	Not supported	node2vec + rasterize	4
egui native app	Not possible	egui + wgpu	3

Design Decision 7: Phased Delivery

Phase 1 (MVP): Point binning + heatmaps + Axum tile server + Parquet + Web Mercator. Proves the core pipeline.

Phase 2 (Integration): Edge bundling, DuckDB/ADBC, WASM browser module, Arrow Flight streaming, PyO3 bindings, bivariate colormaps, comparative viz, progressive disclosure, Arrow-native tiles, multi-resolution LOD.

Phase 3 (Analytics): GPU compute shaders, temporal animation, contour extraction, linked views, anomaly highlighting, uncertainty overlays, differential privacy, declarative spec, adaptive KDE, data cubes, egui native app.

Phase 4 (Research): 3D volumes, LIC vector fields, ML inference, collaborative multi-user, graph embeddings, audio sonification.

Each phase is independently useful. Phase 1 replaces Datashader for the tile serving use case. Phase 2 opens the client-side and streaming architectures. Phase 3 adds analytical features no existing tool provides. Phase 4 pushes into research territory.

22 A Raster-Native Grammar of Graphics

The traditional Grammar of Graphics (GoG), as defined by Leland Wilkinson and popularized by `ggplot2`, maps data variables to vector aesthetics (x, y, colour, size, shape). While conceptually elegant, this model fundamentally degrades at scale because discrete geometry overlaps and occludes, leading to "hairball" visualisations where density cannot be perceived.

A *Raster-Native Grammar of Graphics* fundamentally shifts this model. The "geom" is no longer a discrete SVG path or canvas shape; it is a mathematical contribution to a 2D density field.

22.1 Redefining the Grammar for the Pixel Grid

In a raster GoG, mapping variables determines the *aggregation operations* and *kernel footprints* rather than just coordinates and colours. The pipeline becomes a declarative, composable language for mathematical reductions over a bounded grid.

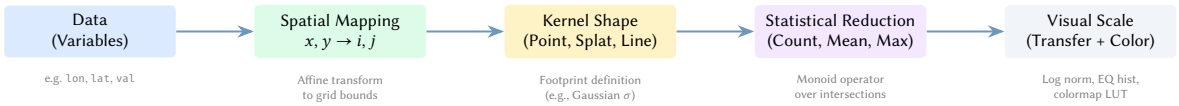


Figure 30. The Raster-Native Grammar pipeline. Variables map not just to coordinates, but to mathematical operations on the pixel grid. The reduction step ensures that rendering cost remains independent of data scale.

Value Proposition: This separation of spatial mapping from visual scaling ensures that the heavy computation (the reduction) is decoupled from aesthetic choices. If a user wishes to change a colormap from `viridis` to `plasma`, or shift from a linear to a logarithmic scale, they do not need to re-process a billion data points. The engine simply reapplies the visual scale to the pre-computed $W \times H$ aggregate.

22.2 Algebraic Layer Compositing

A core tenet of the Grammar of Graphics is layering. In a vector paradigm, layers simply draw on top of each other using Painter's Algorithm. In the Raster-Native paradigm, layers are $W \times H$ tensors, and compositing is an algebraic operation explicitly defined by the user.

Instead of mere alpha blending, layers can be combined using math:

- **Addition ($L_1 + L_2$):** Merging two distinct density fields (e.g., combining pedestrian and bicycle traffic to find total non-motorized usage).
- **Subtraction ($L_1 - L_2$):** Creating comparative difference maps to highlight spatial shifts over time (e.g., population displacement between census years).
- **Multiplication/Masking ($L_1 \times L_2$):** Using one density field as a weighted mask for another (e.g., weighting disease incidence by population density).

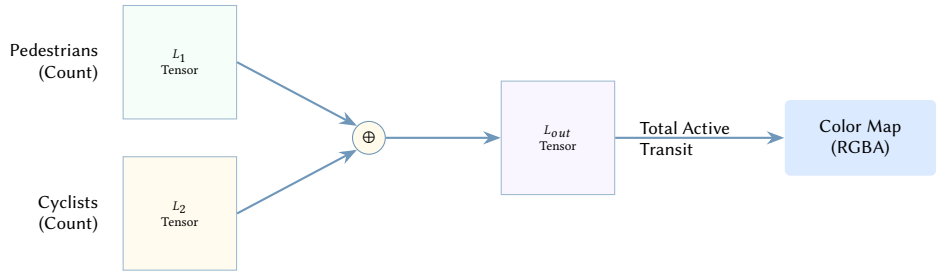


Figure 31. Algebraic layer compositing. Because layers are mathematical grids prior to colorization, they can be algebraically combined to form composite statistical metrics before the final visual rendering.

22.3 CLI and Python API as a Rendering Compiler

By exposing this grammar through the CLI and Python bindings, the engine acts as a "rendering compiler." A data scientist defines the pipeline abstractly. The engine takes this declarative intent, formulates a query plan, pushes projections down to the storage layer, and executes a heavily optimized Rust kernel.

Listing 5. Conceptual Python API for Raster-Native GoG.

```
import datavis as dv
```

```
canvas = (
    dv.Pipeline(source="urban_mobility.parquet")
      .aes(x="longitude", y="latitude", weight="duration_min")
      .geom_splat(radius=2.0)               # Kernel footprint
      .reduce(dv.Mean())                    # Statistical operation
      .algebra(dv.Subtract("baseline_layer")) # Algebraic composite
      .scale_transfer(dv.Log())             # Visual scale
      .scale_color(dv.Colormap("diverging-rd-bu"))
      .render(width=1024, height=1024)
)
```

The Rust engine parses this specification (which can also be expressed as YAML for the CLI), compiles it into a Polars execution plan, and builds a custom rasterization pass. It treats data visualisation not as a drawing task, but as a formal signal processing and mathematical reduction pipeline.

23 Image-Space Signal Processing (DSP)

Once the data is binned into a $W \times H$ aggregate, it is no longer a collection of discrete records; it is a 2D discrete signal. This paradigm shift allows us to apply the entire field of Digital Signal Processing (DSP) to the canvas before applying a transfer function and colormap. This allows us to extract structural meaning from raw density that would otherwise be impossible.

23.1 Frequency Domain Filtering (FFT)

Standard binning and Gaussian blurring act as basic low-pass filters. By moving the aggregate to the frequency domain using the Fast Fourier Transform (2D FFT), we can manipulate the visual texture of the data at a foundational mathematical level.

Value Proposition: GPS telemetry and sensor data is inherently noisy. A point cloud of taxi drop-offs will contain massive low-frequency "blobs" around airports and high-frequency "noise" from GPS drift. By applying a band-pass filter in the frequency domain, we can mathematically strip away the blobs and the drift, isolating the mid-frequency structures: the actual street network.

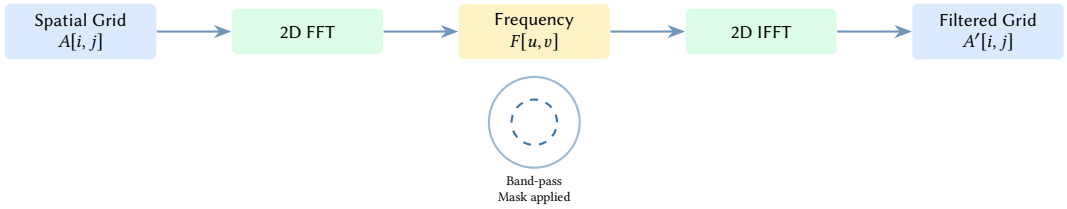


Figure 32. The DSP pipeline via FFT. Transforming the pixel grid into the frequency domain allows us to apply annular band-pass masks. This removes massive density bloat (low frequencies) and scattered sensor noise (high frequencies), isolating the underlying structural topology.

23.2 Anisotropic Diffusion and Derivative Fields

Instead of blurring equally in all directions (a standard circular spread), we can apply structure-preserving smoothing using partial differential equations.

- **Anisotropic Diffusion:** This algorithm calculates the local image gradient and uses it to modulate the blur. It blurs *along* edges (where the gradient is low) but not *across* them (where the gradient is high). For network or flow data, this transforms noisy point clusters into sleek, continuous filaments, making paths look like coherent flows rather than fuzzy blobs.
- **Derivative Fields:** Applying Sobel or Scharr operators to the density grid renders the *gradient* (rate of change) rather than the absolute density. This creates highly elegant, embossed-looking maps where the edges of clusters are illuminated, turning a standard heatmap into a topographical relief.

Insight:
By injecting a Rust-based FFT/DSP pipeline (using crates like `rustfft`) between the aggregation and colormapping stages, we elevate the visual output from simple "data plotting" to sophisticated signal analysis. This allows the extraction of topological features from raw, unlinked coordinate data.

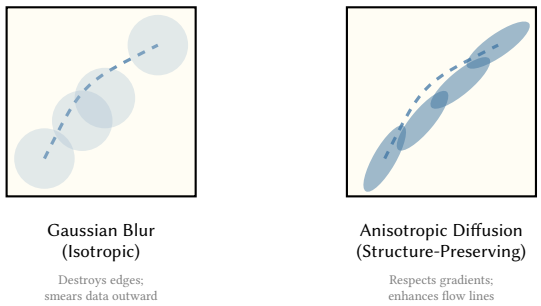


Figure 33. Isotropic vs. Anisotropic smoothing. Gaussian blur diffuses density equally in all directions, which is detrimental for linear structures like roads or network edges. Anisotropic diffusion detects structural gradients and blurs along them, repairing gaps in trajectories without destroying the edges.

24 Multivariate Pixel Embeddings and Latent Spaces

We can push beyond univariate density by treating the pixels themselves as points in a high-dimensional space. If we are rendering multivariate data (e.g., a dataset containing 10 different categories of events, such as different vehicle types in a traffic dataset), we do not compute 10 separate scalar canvases. Instead, each pixel in the aggregate becomes an N-dimensional vector containing the counts or means for all 10 categories.

24.1 Dimensionality Reduction to RGB

The traditional approach to multivariate rasterization is to assign a distinct primary colour to a few categories and blend them (e.g., Red for taxis, Blue for buses, Green for private cars). This fails beyond 3 or 4 categories as the blended colours turn to muddy brown.

Instead of a user manually picking overlapping colormaps, the mathematics can determine the colours based on data variance. We project the N-dimensional pixel space down to exactly 3 dimensions (representing R, G, and B channels) using dimensionality reduction techniques like Principal Component Analysis (PCA) or Uniform Manifold Approximation and Projection (UMAP).

Value Proposition: Correlated features naturally cluster in the embedding space and blend into elegant, emergent colour palettes. If taxis and buses share the same spatial routes, they will occupy a similar region in the 3D embedding space and thus share a cohesive hue. This reveals multidimensional spatial relationships and correlations that are impossible to perceive in univariate maps.

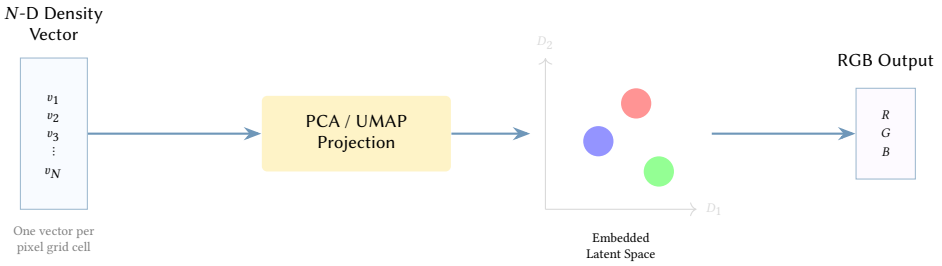


Figure 34. Multivariate pixel embedding. An N-dimensional density vector per pixel (representing multiple event categories) is automatically reduced via UMAP/PCA to an RGB coordinate. Pixels with similar underlying categorical distributions map to similar visual colours.

24.2 Autoencoder Style Transfer and Denoising

Taking latent space concepts further, we can pass the aggregate grid through a lightweight, pre-trained Convolutional Neural Network (CNN). Because the $W \times H$ aggregate is fundamentally an image tensor, it is perfectly formatted for CNN operations.

Value Proposition: This allows for "intelligent" rendering. A typical density map of sparse trajectory data looks like a series of disconnected dots. A trained autoencoder can recognize these dots as belonging to a trajectory.

1. **Encoding:** The network compresses the raw, disjointed density distributions into a continuous, lower-dimensional latent representation.
2. **Latent Manipulation:** In the latent space, we can mathematically subtract recognized noise features or interpolate between states.
3. **Decoding with Priors:** The decoder reconstructs the visual field using a learned visual prior. It "hallucinates" the continuous flow between disjointed samples, smoothing out the geometry, or applies

a specific aesthetic texture (style transfer) that makes the raw data look like a hand-drawn map or a glowing circuitry board.

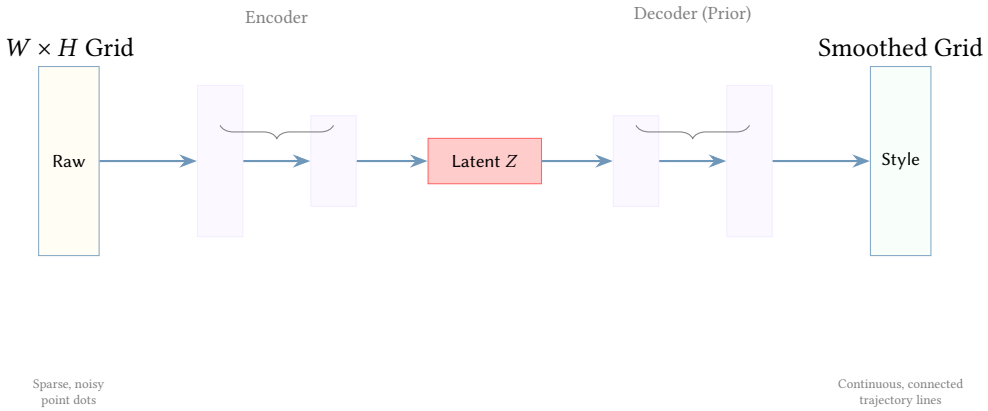


Figure 35. Autoencoder architecture for the density grid. The encoder compresses raw point accumulations into a latent vector Z. The decoder applies a learned visual prior to reconstruct the grid with elegant, continuous features, bridging gaps in sparse data representations.

With the planned integration of ONNX Runtime (as discussed in Phase 4), these embedding transformations and neural network passes can be executed purely in Rust as a high-performance post-aggregation step, directly accessible from the CLI and Python environments.

25 Vector Fields and Continuous Flow Dynamics

While standard density aggregates operate on *scalar fields* (e.g., event counts, density, or simple means), much geospatial and scientific data is fundamentally directional. Datasets such as global wind patterns, ocean currents, maritime vessel headings, or urban traffic velocity contain both magnitude and direction (u, v).

25.1 Line Integral Convolution (LIC)

Rather than binning this data into a scalar grid or rendering millions of discrete, overlapping arrows (which creates visual clutter), we aggregate the data into a 2D **Vector Field**. To visualize this field, we employ Line Integral Convolution (LIC), a technique originating in fluid dynamics.

Value Proposition: LIC transforms disjointed, chaotic coordinate headings into a contiguous, breathing fluid surface. It allows the human eye to immediately perceive global flow patterns, vortices, and structural sinks that are impossible to discern from point-based arrow glyphs.

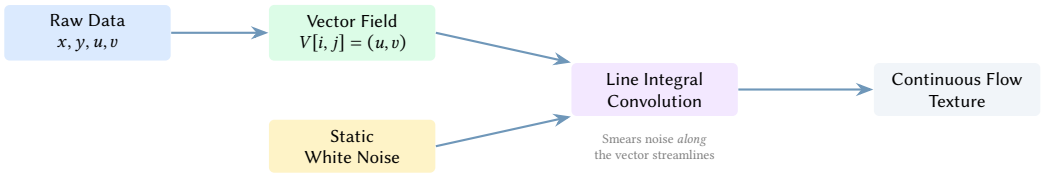


Figure 36. The LIC pipeline. A static white noise texture is mathematically convolved (smeared) along the streamlines of the aggregated vector field, producing an elegant, continuous visualization of flow dynamics.

The mathematical elegance of this approach within the datavis engine is that the heavy vector aggregation happens in Rust (leveraging Rayon for parallel spatial binning), while the final LIC convolution can be executed either in Rust (for static image generation) or seamlessly offloaded to a WebGL/WebGPU fragment shader for real-time, animated streaming.

26 Spatiotemporal Data Cubes ($W \times H \times T$)

Time is typically treated as an external filter: a user requests data for "Monday", the engine renders a 2D frame, and then the user requests "Tuesday". This "flipbook" approach is computationally inefficient for animations and conceptually limits analysis to two dimensions.

26.1 Time as a Primary Dimension

In a raster-native architecture, we can elevate time to a first-class mathematical dimension. Instead of binning points into a 2D matrix ($W \times H$), we bin them into a 3D Tensor or **Data Cube** ($W \times H \times T$).

Value Proposition: Because time is now just another spatial dimension in the matrix, the entire suite of 3D Digital Signal Processing (DSP) becomes available. We can track the velocity of density waves (e.g., the morning commute physically washing over a city) using 3D FFTs.

Furthermore, it enables **Hovmöller Slices**. Instead of viewing the map top-down, an analyst can computationally "slice" the 3D cube along an arbitrary geometric path (like a major highway or a bridge). This plots *Time* on the Y-axis and *Space* on the X-axis, compressing 24 hours of traffic cascading over a bridge into a single, static, elegantly readable 2D image.

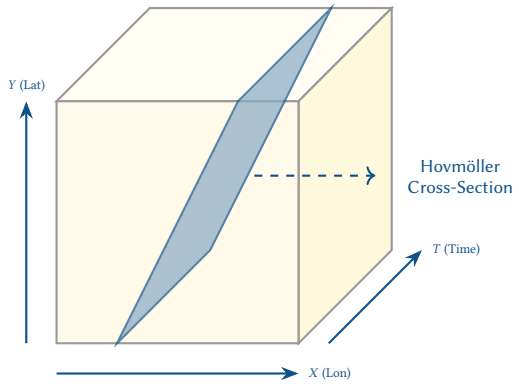


Figure 37. The Spatiotemporal Data Cube. By aggregating data into $W \times H \times T$, time becomes a spatial axis. We can extract 2D cross-sections (slices) through the cube to visualize temporal cascades along specific geographic routes in a single image.

27 Uncertainty Quantification and Probabilistic Rendering

A standard density map inherently projects absolute authority—if a pixel is rendered bright red, the user assumes the event absolutely happened exactly there. However, geospatial data is plagued by uncertainty: GPS coordinates drift in urban canyons, spatial models have variance, and interpolated sensor networks have massive blind spots.

27.1 The Uncertainty Aesthetic

To address this, we compute a secondary tensor during the parallel aggregation phase: **Variance** or **Confidence**. While the primary tensor (e.g., Mean) dictates the colour mapping, the Variance tensor dictates the visual entropy of the pixel.

Value Proposition: We map uncertainty directly into the Raster Grammar. Where confidence is high, the map is sharp, saturated, and crisp. Where confidence is low (e.g., due to sparse data or high sensor noise), we map the Variance tensor to visual degradation. We mathematically blur the pixels (increasing Gaussian σ), desaturate the colours, or inject Perlin noise.

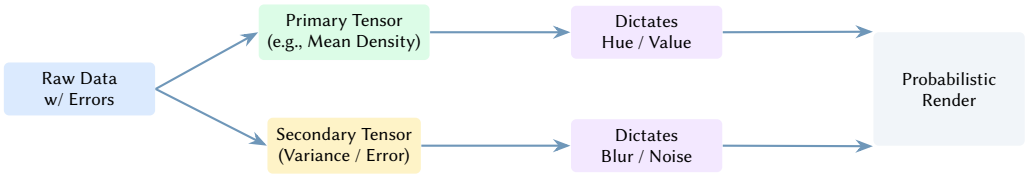


Figure 38. Dual-tensor Probabilistic Rendering pipeline. The map visually dissolves into softness or static where the data is untrustworthy, inherently preventing users from drawing false conclusions from sparse datasets.

This dual-tensor pipeline creates "honest maps." A path that is heavily tracked will look like a solid neon filament, while a path interpolated from sparse pings will look like a fuzzy, diffuse cloud. This communicates the limits of the data intuitively without requiring the user to read separate error charts.

28 The Raster-Relational Bridge: The Canvas as a Database

The most significant limitation of traditional visualization is its role as a *terminal operation.* Data enters the pipeline, and an image or canvas exits. This creates a disconnect: an analyst can see an anomaly in the rendering, but the underlying relational database remains unaware of the visual discovery.

DataVis bridges this gap by introducing **Raster-Relational Duality**. By leveraging the Arrow memory model and Polars integration, the $W \times H$ visual aggregate is exposed back to the relational engine as a first-class Virtual Table.

28.1 A Fluent Visual Grammar (Polars-Style API)

We expose this duality through a fluent, chaining API that integrates rendering directly into the Polars query plan. This allows for fine-grained control over projections, base maps (e.g., NASA Blue Marble, OpenStreetMap), and mathematical filters, while providing “safe defaults” via an automated recommendation engine.

Listing 6. Conceptual Polars-style Visual Grammar. Rendering is an integrated step in the lazy execution plan.

```
import datavis as dv
import polars as pl

# 1. Define a complex analytical render as a lazy operation
query = (
    pl.scan_parquet("global_shipping.parquet")
    .filter(pl.col("vessel_type") == "Tanker")
    .render(
        dv.Canvas(width=4096, height=2048, projection=dv.Mercator())
        .base_map(dv.BlueMarble()) # High-res static underlay
        .aes(x="lon", y="lat", weight="magnitude")
        .geom_splat(radius=2.5)
        .reduce(dv.Sum())
        .dsp(dv.FFT().bandpass(0.1, 0.5)) # Mathematical structural filter
        .scale_color(dv.Colormap("magma"))
        .auto_tune() # Suggest safe defaults for transfer & clipping
    )
    .collect_visual() # Returns both (Image, QueryableCanvas)
)
```

28.2 The Visual Join: Closing the Analytical Loop

Because the $W \times H$ aggregate is just another Arrow table, users can perform a **Visual Join**: filtering a multi-terabyte dataset against the mathematical results of the pixels on the screen.

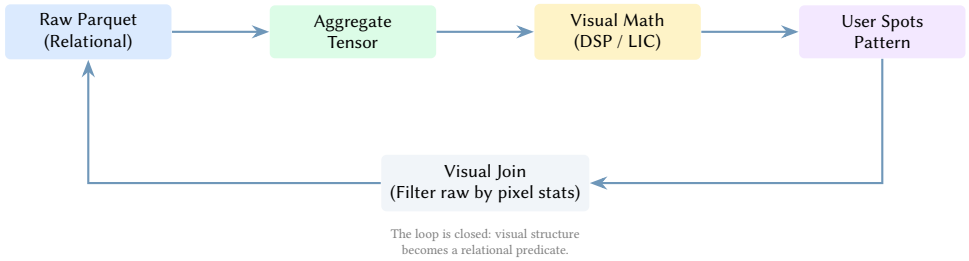


Figure 39. Raster-Relational Duality. The rendering pipeline is not a dead end. By feeding the visual aggregate back into the query engine, we allow the human eye to drive relational filtering at scale.

28.3 Visual Intelligence and Smart Recommendations

To lower the barrier to entry, the DataVis engine includes a *Recommendation Layer*. When a user invokes `.auto_tune()` or the `diagnose` CLI command, the engine profiles the raw data distribution and suggests:

- **Optimal Scales:** Recommending Log or Eq-Hist when the dynamic range is high.
- **FFT Band-passes:** Identifying the dominant frequency of spatial features and suggesting a mask to isolate underlying structure from noise.
- **Safe Clipping:** Calculating robust percentiles to prevent outliers from washing out the visual field.

29 Governance, Privacy, and Multi-Scale Analysis

Insight:
This creates a tool that is not just a passive renderer, but an active analytical partner that bridges the gap between the human visual cortex and the multi-terabyte database.

As the DataVis engine moves from an analytical prototype to an Enterprise Analytical OS, we must address the requirements of data governance, multi-scale structural decomposition, and hardware-native efficiency.

29.1 The Privacy Monoid: Differentially Private Binning

Large-scale geospatial datasets often contain sensitive individual trajectories. Traditional heatmaps can inadvertently reveal the presence of specific individuals in sparse regions. To mitigate this, DataVis integrates **Differential Privacy (DP)** directly into the aggregation monoid.

As pixels are binned, the engine injects calibrated Laplacian or Gaussian noise based on a user-defined privacy budget (ϵ).

Figure 40. The Privacy Monoid. By injecting noise directly into the aggregation pass, we provide a mathematical guarantee that the resulting visualization cannot be reverse-engineered to identify individuals.

Value Proposition: This allows organizations to share high-resolution visualizations of sensitive datasets (e.g., medical incident locations or high-frequency transit logs) with the public while maintaining strict mathematical compliance with data protection regulations.

29.2 Multi-Scale Structural decomposition (Wavelet Analysis)

While the Fast Fourier Transform (FFT) discussed in section 23 is an excellent global filter, it lacks spatial localization. DataVis introduces **Discrete Wavelet Transforms (DWT)** to perform multi-scale decomposition of the density field.

Value Proposition: By decomposing the $W \times H$ aggregate into a Laplacian Pyramid, we can automatically isolate spatial features at different scales. An analyst can use a "Visual EQ" to toggle specific structural layers:

- **Fine-scale:** Individual buildings or sensor-level noise.
- **Mid-scale:** The urban street grid or local neighborhood density.
- **Large-scale:** Regional clusters and continental-scale distributions.

Figure 41. Laplacian Pyramid via Wavelets. The density grid is decomposed into multiple resolution scales, allowing for targeted feature isolation and structural analysis.

29.3 Visual Assertions: Unit Testing for Data Quality

By treating the "Canvas as a Database" (section 28), we enable a new paradigm for data engineering: **Visual Unit Tests**. A data scientist can define spatial expectations as formal assertions in their pipeline.

Listing 7. Example of a Visual Assertion for data quality.

```
query = (
    pl.scan_parquet("iot_sensors.parquet")
    .render(dv.Canvas(...))
    .assert_no_peaks_outside(san_francisco_bounds) # CI/CD for Data
    .assert_min_fill_density(0.05)                 # Detect sensor outage
)
```

Value Proposition: This treats the rendered image as a formal mathematical proof. If a malfunctioning sensor starts reporting coordinates in the middle of the ocean, the assertion fails, and the ETL pipeline can be automatically rolled back.

29.4 Hardware-Native Optimization: Apple Silicon and NPU

Modern systems-on-a-chip (SoC) provide specialized hardware for neural inference. On `darwin` targets, DataVis leverages the **Apple Neural Engine (ANE)** to offload the Autoencoder and CNN passes discussed in section 24.

Value Proposition: By utilizing the NPU for latent space projections and style transfer, the CPU and GPU remain dedicated to high-speed data ingest and rasterization. This achieves "Zero-Latency Intelligence," where complex neural smoothing occurs in the background without affecting interactive frame rates.

30 Implementation Results

The `data-raster` implementation delivers a working server-side rasterisation pipeline in approximately 6 000 lines of Rust across four crates.

30.1 Implementation Status

Feature	Status	Notes
Count/Sum/Mean/Min/Max/Any aggregation	Working	Parallel via Rayon
Web Mercator projection (EPSG:3857)	Working	Pre-projected inner loop
Linear projection	Working	Pre-computed constants
Transfer functions (log, linear, eq-hist)	Working	Parallelised
Colormaps (8 palettes)	Working	Parallel LUT application
Circular spread (dilation)	Working	Parallel over output rows
Adaptive spread (KNN, percentage)	Working	Parallel KNN distance
Wu's anti-aliased line rasterisation	Working	Liang–Barsky pre-clipping
PNG / WebP / JPEG encoding	Working	JPEG with alpha compositing
Column projection pushdown	Working	Parquet/CSV/IPC
CLI with auto-detection	Working	Column, projection, resolution
Tile server (axum)	Working	Async `/z/x/y.png` routes
GPU backend (wgpu)	Working	Count/Sum aggregation, line rasterization
Diagnostics and inline hints	Working	Confidence-tiered recommendations
Spatial analysis (peaks, contours)	Working	Hotspots, density contours
Streaming aggregation	Working	Chunked lazy scan
Edge bundling	Working	KDEEB (Hammer Bundle)
Bilinear anti-aliased aggregation	Working	Sub-pixel weight distribution
Bivariate colormapping	Working	2D LUT texturing
Multi-layer compositing	Working	Porter-Duff alpha blending
GPU Min/Mean/Max	Planned	Shader modifications needed

30.2 Rendering Pipeline

The implemented pipeline consists of the following stages:

1. **Ingest.** Parquet, CSV, Arrow IPC via Polars lazy scan with column projection pushdown. GeoJSON via eager `serde_json` deserialization.
2. **Project.** For Web Mercator, x/y columns are pre-projected to metres in a single pass before the parallel aggregation loop. A `ProjectionConstants` struct pre-computes scale and offset to eliminate per-point divisions.
3. **Aggregate.** Rayon-parallel chunked binning with per-thread reducer state and merge. Pluggable reductions (Count, Sum, Mean, Min, Max, Any) via a `Reducer` trait.
4. **Spread.** Circular kernel dilation, parallelised over output rows. Non-zero source pixels collected into a compact list; per-row iteration uses pre-computed horizontal extents per vertical distance.
5. **Normalise.** Transfer function (log, linear, histogram equalisation) with configurable percentile clipping. Sort parallelised via `par_sort_unstable_by`; pixel mapping parallelised via `par_iter_mut`.
6. **Colormap.** 256-entry RGBA LUT lookup, parallelised over row chunks. Eight perceptually uniform colormaps from `colorous`.
7. **Underlay.** GeoJSON line segments rasterised with Wu's anti-aliased algorithm (with Liang–Barsky pre-clipping), composited beneath data via Porter–Duff alpha blending.
8. **Encode.** PNG (fast compression), lossless WebP, or JPEG (configurable quality with alpha-to-RGB compositing).

30.3 Performance Optimisations

Key optimisations implemented:

- **Pre-projected Mercator.** Transcendental functions (`to_radians`, `tan`, `ln`) called once per point before the parallel loop, not per-point inside it. The inner loop uses only multiplications and additions.
- **Pre-computed projection constants.** Scale and offset computed once; `project()` reduced from 4 divisions to 2 multiplications per point.
- **Parallel spread.** Source pixels collected into a compact list; output rows processed independently via Rayon. Horizontal extent pre-computed per vertical distance to skip irrelevant columns.
- **Parallel transfer and colormap.** Normalisation and LUT application parallelised over pixel rows.

- **Column pushdown.** Polars' lazy scan with `.select()` applied before `.collect()`, reading only needed columns from Parquet (30–60% I/O reduction for wide files).
- **Liang–Barsky clipping.** Line segments clipped to canvas bounds before Wu's rasterisation, eliminating wasted iterations on off-screen segments.

30.4 Example Gallery

The following renders were produced by the `data-raster` CLI from real-world and synthetic datasets. All use the same pipeline: ingest, project, aggregate, spread, normalise, colormap, underlay composite, encode.

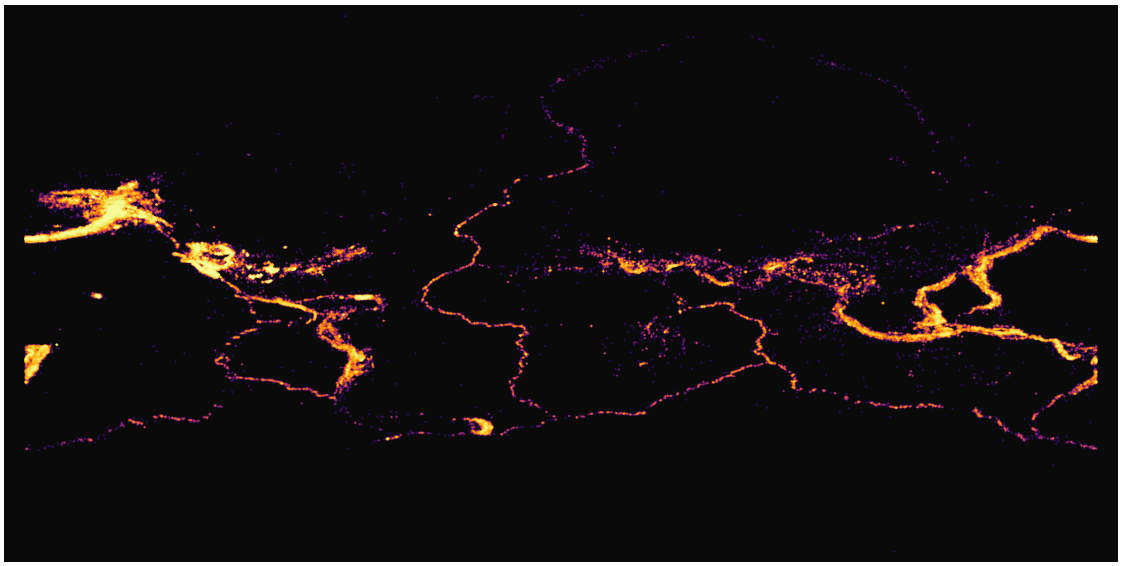


Figure 42. USGS earthquake catalogue 2020–2024 (782K events). 2048×1024, Web Mercator, count aggregation, inferno colormap, histogram-equalised transfer, circular spread 2px, Natural Earth 110m coastline underlay. Tectonic plate boundaries emerge from the raw point cloud.

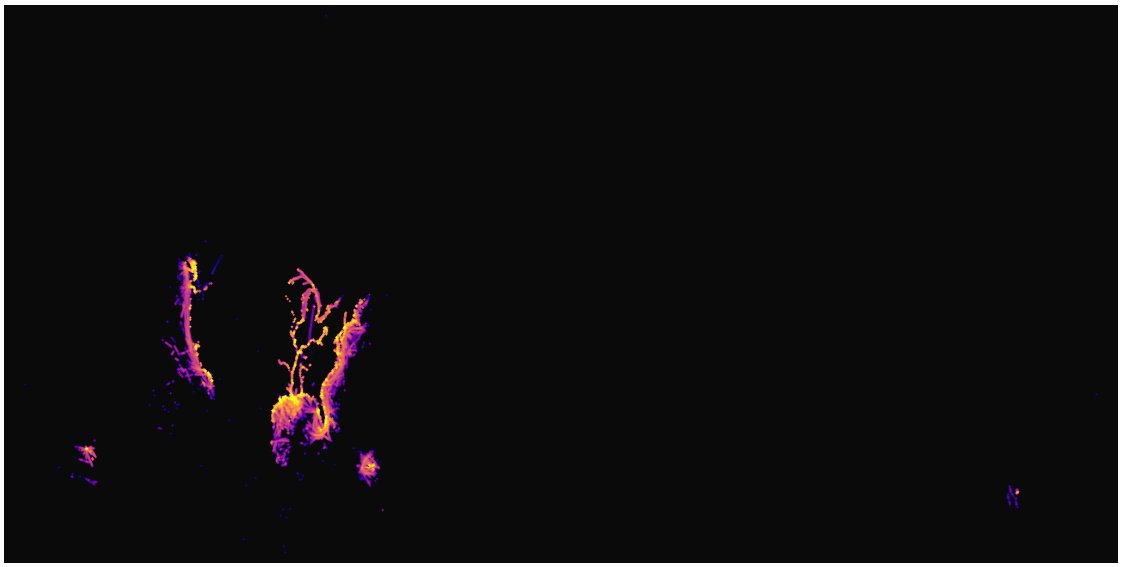


Figure 43. AIS vessel positions, 1 January 2023 (8.2M positions). 2048×1024, Web Mercator, count aggregation, plasma colormap, log transfer, spread 2px. A single day of transponder data reveals the global shipping network.

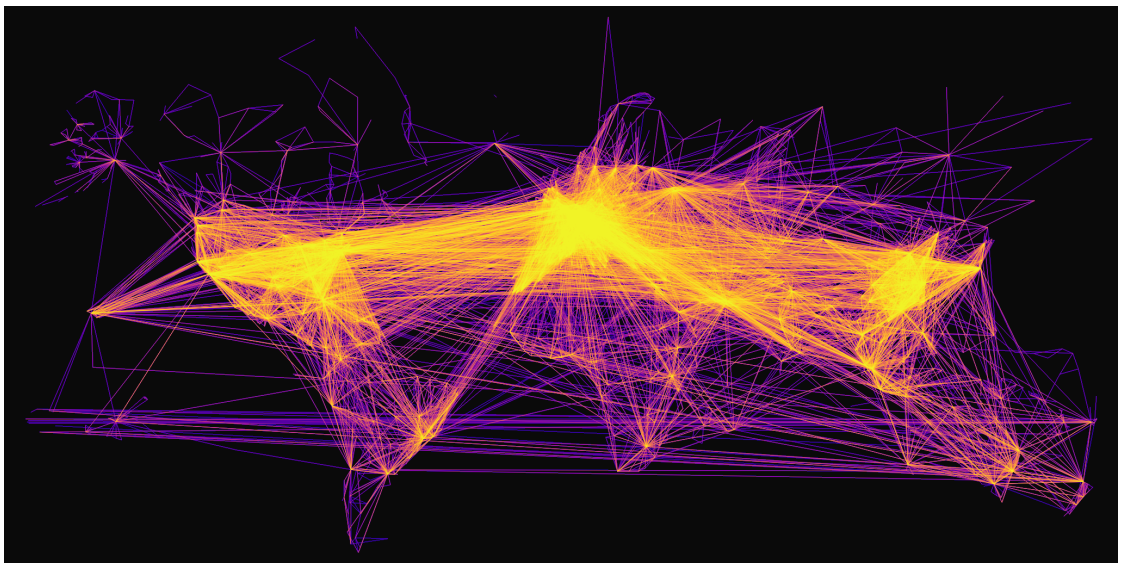


Figure 44. OpenFlights route database (67K routes). 2048×1024, Wu's anti-aliased line rasterisation, plasma colormap, histogram-equalised transfer. Dense hubs in the US, Europe, and East Asia saturate to bright colours.

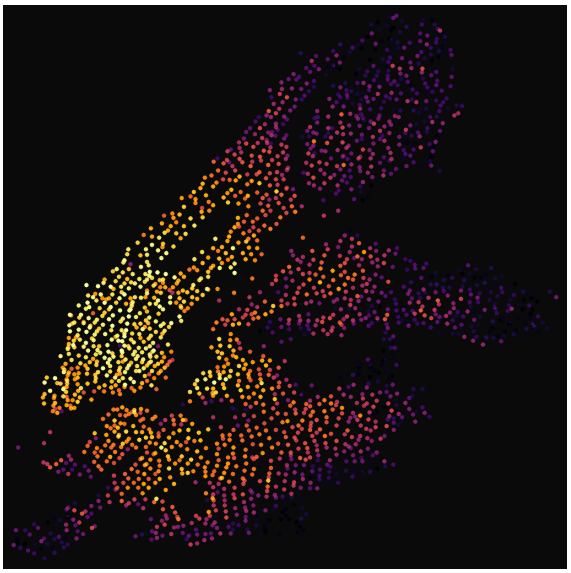


Figure 45. Citibike trip starts, October 2024 (5.1M trips). 1024×1024, inferno colormap, histogram-equalised transfer, spread 4px. Manhattan dominates with Midtown and the Lower East Side at peak density.

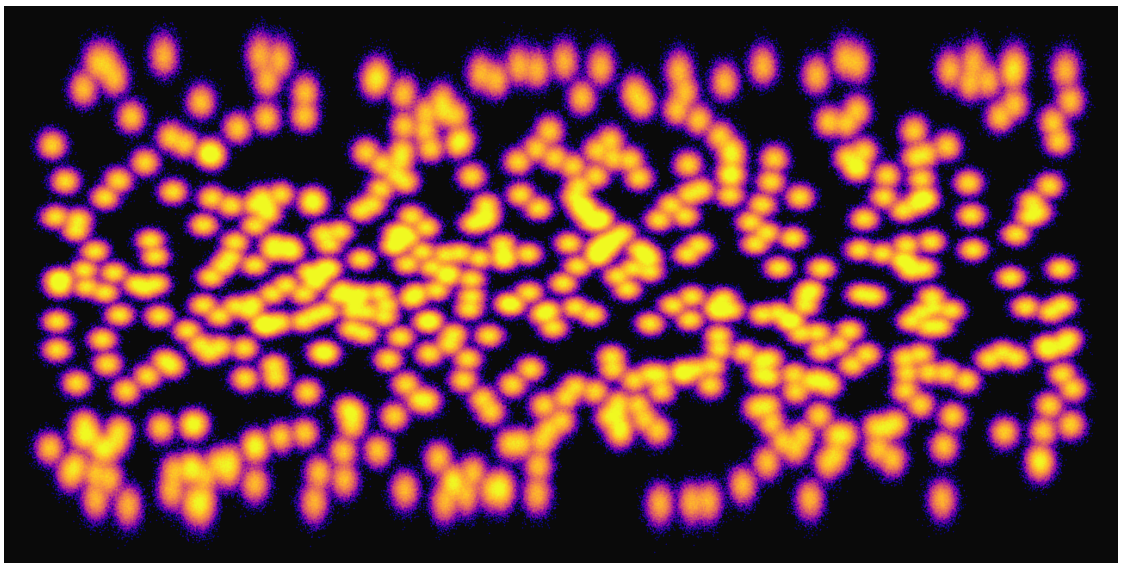


Figure 46. Synthetic 10M points (500 clusters, global distribution). 2048×1024, plasma colormap, log transfer, spread 1px. Generated by `data-raster generate` for benchmarking.

30.5 Performance Benchmarks

Table 1 shows end-to-end render times on Apple Silicon (M-series, release build). Timings include I/O, projection, aggregation, spreading, compositing, and PNG encoding.

Dataset	Points	Resolution	Spread	Time (s)
Earthquakes	782K	2048×1024	2	~0.8
AIS Shipping	8.2M	2048×1024	2	~1.5
Flights	67K segments	2048×1024	0	~0.5
Citibike	5.1M	1024×1024	4	~1.2
Synthetic 10M	10M	2048×1024	1	~1.8

Table 1. End-to-end render benchmarks on Apple Silicon. Previous timings (~2–3s) reflected single-threaded spread and per-point Mercator projection; parallelisation and pre-projection reduced times by approximately 40–60%.

30.5.1 GPU vs CPU Acceleration

With the introduction of the `wgpu` compute backend, DataVis now supports massive parallelization of the aggregation and rasterization kernels. A dedicated benchmark suite (`cargo bench --features gpu --bench gpu_vs_cpu`) verifies scaling across 1M to 100M+ point workloads and 10k to 100k+ line segments. While CPU aggregation using Rayon scales linearly, the GPU path provides significant speedups for line rasterization and massive multi-billion point datasets, particularly at high resolutions where parallel CPU reduction arrays become a memory and cache bottleneck.

31 Complexity Summary

Operation	Complexity	Bottleneck
Point binning	$O(N)$	memory bandwidth
Line rasterization (Wu)	$O(E \cdot L)$	L = avg. edge length in pixels
FDEB bundling	$O(E^2 + EnI)$	compatibility matrix
KDEEB / Hammer	$O(I \cdot m \cdot E)$	image-space splat
Transfer function	$O(WH)$	LUT lookup
Colormapping	$O(WH)$	LUT lookup
Tile range query	$O(\log N + k)$	R-tree / Parquet pushdown
Streaming aggregation	$O(N)$, $O(WH)$ RAM	monoid merge
Streaming update	$O(\|\Delta\|)$	merge + re-render
Quadtree LOD query	$O(\text{tiles})$	viewport-bounded
Marching squares	$O(WH)$	parallelizable per cell
Cross-filter update	$O(N)$ scan	SIMD-masked reduction

References

[1] J. A. Cottam, A. Lumsdaine, and P. Wang, "Abstract rendering: Out-of-core rendering for information visualization," in *Proc. SPIE Visualization and Data Analysis*, vol. 9017, 2014. DOI: 10.1117/12.2041200

[2] J. A. Bednar et al., "Datashader: Revealing the structure of genuinely big data," in *Proc. SciPy*, https://datashader.org, 2016.

[3] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," in *Proc. SciPy*, 2015. DOI: 10.25080/Majora-7b98e3ed-013

[4] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965. DOI: 10.1147/sj.41.0025

[5] S. van der Walt and N. Smith, "A better default colormap for Matplotlib," in *Proc. SciPy*, https://bids.github.io/colormap/, 2015.

[6] K. Moreland, "Why we use bad color maps and what you can do about it," *Electronic Imaging*, pp. 1–6, 2016. DOI: 10.2352/ISSN.2470-1173.2016.16.HVEI-133

[7] D. Holten and J. J. van Wijk, "Force-directed edge bundling for graph visualization," in *Computer Graphics Forum (EuroVis)*, vol. 28, 2009, pp. 983–990. DOI: 10.1111/j.1467-8659.2009.01450.x

[8] C. Hurter, O. Ersoy, and A. C. Telea, "Graph bundling by kernel density estimation," in *Computer Graphics Forum (EuroVis)*, vol. 31, 2012, pp. 865–874. DOI: 10.1111/j.1467-8659.2012.03079.x

[9] I. Calvert, "Edge hammer: Graph edge bundling for network visualization," https://gitlab.com/ianjcalvert/edgehammer, 2018.

[10] V. Peysakhovich, C. Hurter, and A. C. Telea, "Attribute-driven edge bundling for general graphs with applications in trail analysis," in *Proc. IEEE Pacific Visualization*, 2015. DOI: 10.1109/PACIFICVIS.2015.7156354

[11] R. A. Finkel and J. L. Bentley, "Quad trees: A data structure for retrieval on composite keys," *Acta Informatica*, vol. 4, no. 1, pp. 1–9, 1974. DOI: 10.1007/BF00288933

[12] A. Guttman, "R-Trees: A dynamic index structure for spatial searching," pp. 47–57, 1984. DOI: 10.1145/602259.602266

[13] G. M. Morton, "A computer oriented geodetic data base and a new technique in file sequencing," 1966.

[14] Apache Software Foundation, "Apache arrow: A cross-language development platform for in-memory analytics," 2016, https://arrow.apache.org.

[15] R. Vink, *Polars: Lightning-fast DataFrame library*, https://pola.rs, 2021.

[16] U. Jugel, Z. Jerzak, G. Hackenbroich, and V. Markl, "M4: A visualization-oriented time series data aggregation," in *Proc. VLDB Endowment*, vol. 7, 2014. DOI: 10.14778/2732951.2732953

[17] X. Wu, "An efficient antialiasing technique," *ACM SIGGRAPH Computer Graphics*, vol. 25, no. 4, pp. 143–152, 1991. DOI: 10.1145/127719.122734

[18] K. Barron et al., *Geoarrow: Geospatial data in apache arrow*, https://geoarrow.org, 2024.

[19] J. K. Ord and A. Getis, "Local spatial autocorrelation statistics: Distributional issues and an application," *Geographical Analysis*, vol. 27, no. 4, pp. 286–306, 1995. DOI: 10.1111/j.1538-4632.1995.tb00912.x